

# 531 업그레이드 트리 — 전체 합본 [2026-09-04 생성]

공식문서/531/업그레이드/ 아래 31개 파일을 폴더 순서(0~7)로 합친 것. 개별 파일이 정본이며, 이 합본은 통독·검색용.

## 【파일】 0-수집목차.md

## 5.3.x "Upgrading from Version 5.1" 수집 목차 [2026-09-02]

- 출처: [ibm.com/docs](https://ibm.com/docs) → IBM Software Hub → **5.3.x** → Upgrading → **Upgrading from Version 5.1**
- 성격: ★런북 Stage 1 의 원본 격 문서 트리 — 런북과 대조해 폐쇄망 분기·누락 절차를 확정하는 용도
- 구조: IBM 목차의 절 = 폴더 / 페이지 = 파일. URL 함께 기록.

### 문서 트리 ↔ 폴더 (5.3.x 전체 목차 기준 — 9/2 확장판)

|                       |                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------|
| 0-정보수집/               | Collecting required information                                                     |
| ✓ 1-허브.md             |                                                                                     |
| ✓ 2-컴포넌트결정.md         | ★스케줄러=실측 자동결정 / ws→ws_runtimes 필수 / B3 심판                                           |
| □ 3-모델이미지결정.md        | Determining which models and optional images (IMAGE_GROUPS 판정)                      |
| ✓ 4-환경변수.md           | OLM_UTILS 아키텍처 / IMAGE_GROUPS / PULL 계정 별도                                          |
| 1-클라이언트/              | Updating client workstations – ✓완료 (4/4)                                            |
| ✓ 1-허브.md             |                                                                                     |
| ✓ 2-cpd-cli갱신.md      | ★워크스페이스 복사 절차 / 14.1.3 삭제 금지 판정 / CPD_CLI_MANAGE_WORKSPACE·OLM_UTILS_LAUNCH_ARGS 정본 |
| ✓ 3-oc갱신.md           | "호환 버전" 요구뿐 – 4.20.24 상위호환 결정 유지                                                    |
| ✓ 4-helm설치.md         | 선택·권장 확정 + 디버깅 용도 명시(--cluster_resources 등)                                         |
| 2-폐쇄망준비/              | ★Preparing to run upgrades in a restricted network – ✓완료 (3/3)                      |
| ✓ 1-허브.md             | 스킵 조건(상시 인터넷이면) – 모비스 해당(스킵 불가)                                                     |
| ✓ 2-olm-utils확보.md    | ★접미(.amd64) 확정 / 문서 자체 파일명 오타 확인 / load 성공 문구                                       |
| ✓ 3-CASE다운로드.md       | Downloading CASE packages                                                           |
| 3-사설레지스트리/            | Preparing to run upgrades from a private registry                                   |
| ✓ 1-허브.md             | 스킵 조건(Entitled 직결만) – 모비스 필수                                                        |
| ✓ 2-미러링-허브.md         | ★중간 레지스트리 운반 3옵션(scp 전송 = 우리 방식) 공식 확인                                              |
| □ 2a-직접미러.md          | Mirroring images directly (모비스 해당 없음 – 판정용만)                                        |
| □ 2b-중간레지스트리.md       | ★Mirroring using an intermediary registry – 1번 스크립트 정본                              |
| ✓ 3-olm-utils-pull.md | ★폐쇄망 최종형 확정: <사설>/copen/cpd/olm-utils-v4: \${VERSION}.amd64 (접미 포함) – env.sh 주석 갱신됨 |
| 4-클러스터준비/             | Preparing your cluster                                                              |
| ✓ 1-허브.md             | ★발견: ④-b·④-c 는 스케줄러 조건부(스케줄러용 리소스)                                                  |

|                     |                                                                                                                            |
|---------------------|----------------------------------------------------------------------------------------------------------------------------|
| ✓ 2-전제소프트웨어.md      | - p.50 원본은 ⑤의 인스턴스파일 가능성<br>모비스 해당 = GPU 조건부 2종만(실측) / RHOAI·MCG·<br>Knative 해당없음 → imageset 의 rhods 재검토 후보                |
| ✓ 3-전역리소스-공유.md     | ★★폐쇄망 퍼즐 해소: "CASE already on workstation" 변형<br>공식 존재 - 로컬 CASE 로 yaml 생성, 런북 p.50 현장 실행 가능                               |
| ✓ 4-폴시크릿-공유.md      | 스케줄러 NS 용 시크릿 - ★런북 누락 후보(p.52 는 2개 NS 만).<br>스케줄러 실측과 한 묶음                                                                |
| ✓ 5-공유컴포넌트.md       | License = 런북 일치(+SUCCESS 대기) / ★apply-scheduler<br>명령 확보 - 런북 누락(스케줄러 조건부 3종 세트 완성)                                        |
| 5-인스턴스준비/           | Preparing to upgrade an instance                                                                                           |
| ✓ 1-허브.md           | ★Events Operator = wxA/wxO 전용 → 모비스 해당없음 확정<br>(런북 누락 후보 ⑧ 해소)                                                             |
| ✓ 2-건강검진.md         | ★4번째 검사(network-connectivity) - 런북에 없음 /<br>폐쇄망은 레지스트리 도달 FAIL 정상 / 기준 90%·350MB/s                                         |
| ✓ 3-전역리소스-인스턴스.md   | ★런북 p.50 원본 확정 + 폐쇄망 변형(CASE 로컬) 재확인<br>- p.50 현장 실행 가능 최종 종결 / Helm 컴포넌트만 생성                                              |
| ✓ 3b-Events오퍼레이터.md | 모비스 해당없음 최종 확정(wxA/wxO 전용) -<br>단 삼성 랩(wxo)엔 유관이라 명령 보존                                                                    |
| ✓ 4-NSS재인증.md       | jq 판별 = 런북 일치 / ★false 대응 절차 전문 확보<br>(show-minimum-rbac → role-binding - 런북은 링크뿐이던 것)                                     |
| ✓ 5-사용자재인증.md       | admin 롤 사용자면 스킵(모비스 해당없음 추정 - 실측) /<br>해당 시 절차 전문 확보(role 2종×2NS + 부여 4회)                                                  |
| ✓ 6-엔타이틀먼트.md       | ⑨판정: 누락 아닌 조건부 - 설치 때 실행 여부만 실측.<br>명령 골격·라이선스명 [사수확인] 확보                                                                  |
| 6-인스턴스업그레이드/        | Upgrading an instance                                                                                                      |
| □ 1-허브.md           |                                                                                                                            |
| ✓ 2-폴시크릿-인스턴스.md    | 런북 p.52 완전 일치 / Helm 용 자격증명이라는 용도 확인                                                                                       |
| ✓ 3-SWH업그레이드.md     | ★9/3 전문 승격(URL 확정) - 런북 p.53~60 원본 + 신규 5건:<br>--preview / storage_tests 해설 / 배치 업그레이드 /<br>health 기대표 / 반자동 활성화 patch 명령  |
| ✓ 4-cpdr서비스갱신.md    | ★B10 뿌리 확정(4갈래 짜깁기) / 런북과 플래그 차이<br>(--cpfs-image-version=latest ↔ prefix 방식) /<br>스케줄러 조건부 4번째(--cpd-scheduler-namespace) |
| 7-마무리-웹훅-모니터/       | 나머지 - ✓완료 (3장 합본)                                                                                                          |
| ✓ 1-웹훅-모니터-원격.md    | 셋 다 조건부(실측 3건) / ★ZEN_VERSION 짝표:<br>5.3.1 P12 → zen 6.4.11 / 원격 로케이션 해당없음 확정                                              |

## 지금까지 굵은 수확

- 스케줄러 = oc get scheduling -A 실측으로 자동 결정 [공식 확정]
- ws 있으면 ws\_runtimes 필수 / list-deployed-components = B3 심판
- cpd-cli 전환 시 워크스페이스 복사 절차·권한(0777) — 현장 절차
- △런북 누락 후보 2: Events Operator 업그레이드 / Stage1 entitlements

수집되면 □ → ✓ + 한 줄 수확 기입.

=====

## 【파일】 0-정보수집/1-허브.md

=====

# 원문 — Collecting information required to upgrade (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading → Upgrading from Version 5.1 → **Collecting required information** URL: [미확인 — 주 소창 복사본 받으면 기입] 문서 갱신일: 2026-03-11

## ★우리 판정:

- ① 허브 페이지 — 본문은 하위 3건 안내뿐. 하위 3건이 알맹이:
  - ① Determining which components to upgrade — ★B3(COMPONENTS 3곳 불일치)의 공식 정보 후보.  
런북 p.34 list-deployed-components 의 출처일 것
  - ② Determining which models/optional images to mirror — 추가 미러(모델 등) 판단.  
모비스는 wx.ai 없음 → 해당 적을 듯 [잠정 — 본문 확인]
  - ③ Updating your environment variables script — env.sh 대조용 정보
- ② 시점 규정: "미러 전 + 인스턴스 업그레이드 준비 전"에 완료하라 —  
런북이 이 순서(실측 후 변수 확정)를 p.34~35 로 옮긴 것과 일치
- ③ 인스턴스마다 반복(Repeat as needed) — 모비스는 단일 인스턴스라 1회

## 본문 (전문)

To successfully upgrade your IBM Software Hub installation, you must have specific information about your environment.

Who needs to complete this task?

- Operations team — determine: which components are installed on the cluster / which components are associated with the instance you want to upgrade.

When do you need to complete this task?

- Repeat as needed — repeat for each instance of IBM Software Hub.
- Ensure that you complete this task before you:
  - Mirror images to your private container registry
  - Prepare to upgrade an instance of IBM Software Hub

Procedure — complete the following tasks:

1. Determining which components to upgrade (5.1→5.3) — determine which components are installed for the instance you want to upgrade.
2. Determining which models and images to mirror to your private container registry — if you use a private registry, you might need to mirror additional images and models (such as foundation models) depending on the services you plan to upgrade and features you plan to enable.
3. Updating your environment variables script — commands use \${VARIABLE\_NAME} variables; ensure your script has correct information for the instance.

=====

## 【파일】 0-정보수집/2-컴포넌트결정.md

# 원문 — Determining which components to upgrade (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading → Upgrading from Version 5.1 → Collecting required information → **Determining which components to upgrade** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=information-determining-which-components-upgrade> [확정] 문서 갱신일: 2026-08-20

### ★우리 판정:

- ① ★스케줄러 규칙 [확정] – "oc get scheduling -A 결과가 있으면 COMPONENTS 에 scheduler 를 포함하라"
  - [사수결정: scheduler] 의 성격이 바뀜: 의견이 아니라 실측 결과에 따라 자동 결정. 설치돼 있으면 포함 확정 / 없으면 제외 확정. 남는 건 실측뿐.
- ② ★ws→ws\_runtimes 규칙 [확정] – "list-deployed-components 가 ws 를 반환하면 COMPONENTS 에 ws\_runtimes 를 반드시 추가하라" (Important) – 우리 COMPONENTS\_531 은 이미 포함 ✓
- ③ list-deployed-components = 컴포넌트 목록의 공식 정보 명령 [확정] – B3(런북 3곳 불일치)의 최종 심판은 이 명령의 실측 결과. 런북 p.34 가 이 페이지의 축약이었음
- ④ 업그레이드 방식 두 갈래: 컴포넌트 개별(런북 방식 – 순서 중요) vs 한꺼번에(cpd-cli 가 순서 자동 처리, 단계 적음). 런북은 개별 방식 채택 – 이유 불명 [참고]
- ⑤ 전 컴포넌트 동일 버전 필수(Important) / 인스턴스별 독립 업그레이드 가능(모비스 단일 이라 무관)

## Upgrade phase (문서의 위치 표시)

Collecting required information ← 여기 → client workstation → restricted network → private registry → prerequisite software → shared cluster components → preparing instance → upgrading instance

## Before you begin (요지)

- 환경변수 스크립트 세팅 후 source 하면 명령 그대로 실행 가능 (Best practice)
- 다중 인스턴스: 인스턴스별 독립 업그레이드 가능, 버전 섞여도 됨 (예: 하나는 5.1.2, 하나는 5.3.1)
- Version support (Important): 한 인스턴스 안의 전 컴포넌트는 동일 버전 필수
- 업그레이드 옵션: 개별(세밀 제어, 단계 많음·순서 중요 — ★오퍼레이터는 어차피 동시 업그레이드) vs 일괄(단계 적음, cpd-cli 가 순서 보장)

## Procedure (전문)

```
# 1. 로그인
${CPDM_OC_LOGIN}
```

```
# 2. 스케줄러 설치 여부
oc get scheduling -A
# 결과 있음 → ★COMPONENTS 에 scheduler 포함할 것 (프로젝트·버전 표시됨)
# 빈 응답 → 미설치

# 3. 인스턴스에 설치된 서비스 목록
cpd-cli manage list-deployed-components \
--cpd_instance_ns=${PROJECT_CPD_INST_OPERANDS}
# ★Important: ws 가 나오면 COMPONENTS 에 ws_runtimes 추가할 것
```

What to do next: → Determining which models and images to mirror

## 관련

- 보정목록 B3 (COMPONENTS 3곳 불일치 — 이 명령 실측이 최종 심판)
- env.sh COMPONENTS\_531 · 로드맵 \$2 실측(스케줄러)

## 【파일】 0-정보수집/4-환경변수.md

# 원문 — Updating your environment variables script (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading → Upgrading from Version 5.1 → Collecting required information → **Updating your environment variables script** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=information-updating-your-environment-variables-script> [확정] 문서 갱신일: 2026-07-15

★우리 판정 (env.sh 대조):

- ① ★OLM\_UTILS\_IMAGE 공식 형태에 아키 접미가 붙음: icr.io/cpopen/cpd/olm-utils-v4:\${VERSION}.amd64  
 런북 p.36 은 접미 없는 :\$VERSION - 차이 발견. 단 "사설 레지스트리면 push 후에 갱신"(Tip)  
 이라 폐쇄망 최종형은 ③-b(Pulling olm-utils) 문서에서 확정할 것 [대조 대기]
- ② IMAGE\_GROUPS 신규 변수 - 모델·선택 이미지 미러용(예: wx.ai 모델). 모비스 해당 여부는  
 ③-b(models to mirror) 문서로 판정 [수집 대기 - ws\_runtimes GPU/NLP 와 관련 가능성]
- ③ 이미지 풀 3형제 공식 형태 확인 - env.sh 와 대조:  
 IMAGE\_PULL\_SECRET(이름 자유·k8s 규칙) ✓ /  
 IMAGE\_PULL\_PREFIX=\${PRIVATE\_REGISTRY\_LOCATION} ✓  
 IMAGE\_PULL\_CREDENTIALS = base64("PULL\_USER:PULL\_PASSWORD") - ★PULL 용 계정 (읽기)이  
 별도 개념: env.sh 엔 PUSH\_USER 만 있음 → PRIVATE\_REGISTRY\_PULL\_USER 추가 후보
- ④ 검증 습관 2개 채택: 문법 검사 bash ./cpd\_vars.sh / 새 셀마다 source 재실행

(Important)

⑤ COMPONENTS 는 "정확한지 확인하라"만 – 목록 자체는 ㉔-a 실측이 정본 (B3 일관)

## 본문 요지 + 명령 (전문 보존)

Editing your environment variables file:

1. VERSION 갱신:

```
export VERSION=5.3.1
```

2. OLM\_UTILS\_IMAGE (라이선스·아키벌):

```
# 표준 (전 고객) – x86-64
export OLM_UTILS_IMAGE=icr.io/cpopen/cpd/olm-utils-v4:${VERSION}.amd64
# (ppc64le / s390x 접미 변형 존재)
# 프리미엄 라이선스 구매 시: cp.icr.io/cp/cpd/olm-utils-premium-v4:${VERSION}.amd64
```

Tip: 사설 레지스트리 클러스터는 push 후에 이 변수를 갱신한다 3. COMPONENTS 확인 (정확한지) 4. 모델·선택 이미지 미리 시:

```
export IMAGE_GROUPS=<comma-separated-list-of-group-names>
```

(기본 미러에 안 딸려오는 모델·선택 이미지는 명시적으로 미러 — 예: watsonx.ai 모델 선택. 상세는 ㉔-b 문서) 5. Image pull configuration 절 추가:

```
export IMAGE_PULL_SECRET=<pull-secret-name>
# 사설 레지스트리 클러스터 (export 명령 수정 금지):
export IMAGE_PULL_CREDENTIALS=$(echo -n
"$PRIVATE_REGISTRY_PULL_USER:$PRIVATE_REGISTRY_PULL_PASSWORD" | base64 -w 0)
export IMAGE_PULL_PREFIX=${PRIVATE_REGISTRY_LOCATION}
# (Entitled Registry 클러스터는 cp:$IBM_ENTITLEMENT_KEY / icr.io)
```

6. 저장 → 문법 검사:

```
bash ./cpd_vars.sh
```

Sourcing: 명령 실행 전 반드시 `source ./cpd_vars.sh` — ★새 bash 프롬프트를 열 때마다 재실행 (Important)

## 관련

- env.sh (우리 정본 — ③의 PULL\_USER 추가 후보·①의 접미 대조 대기)
- ①-a 컴포넌트결정 · ①-b 모델/이미지(수집 대기) · 런북 p.35(B3 함정)

## 【파일】 1-클라이언트/1-허브.md

# 원문 — Updating client workstations (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading → Upgrading from Version 5.1 → **Updating client workstations** URL:  
<https://www.ibm.com/docs/en/software-hub/5.3.x?topic=51-updating-client-workstations> [확정] 문서 갱신일:  
 2026-03-11

### ★우리 판정:

- ① 허브 — 하위 3건이 알맹이: cpd-cli 갱신 / oc CLI 갱신 / Helm 설치.  
 런북 p.13(다운로드)·p.34(클라이언트 갱신)의 원본 격.
- ② 우리 진행상황 대조: cpd-cli 14.3.1 ✓(확보·풀림) / oc 4.20.24 ✓(tools)  
 / helm 3.19.0 ✓(tools) — 3건 다 이미 충족. 하위 페이지는  
 버전 규칙 확인용으로만 수집하면 됨 (oc 는 "클러스터와 호환되는 판" 요구  
 — 4.20.24 클라이언트 상위호환 결정과 대조)
- ③ Helm 의 공식 용도 명시: "업그레이드 이슈 디버깅을 쉽게" —  
 런북 p.13 "(선택)" 표기의 근거

## 본문 (전문)

Before you can upgrade, ensure the workstation has supported versions of: the IBM Software Hub CLI (cpd-cli), the OpenShift CLI (oc), and the Helm CLI.

Who: All administrators — any user involved in upgrading needs access to a client workstation with supported CLI versions. When: Repeat as needed — at least one workstation, repeat for multiple.

### Procedure:

1. Updating the IBM Software Hub CLI — correct version of cpd-cli and the olm-utils-v4 image.
2. Updating the OpenShift CLI — oc version compatible with your cluster.
3. Installing the Helm CLI — to make it easier to debug upgrade issues.

## 【파일】 1-클라이언트/2-cpd-cli갱신.md

# 원문 — Updating the IBM Software Hub CLI (5.1→5.3)

## [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Updating client workstations → **Updating the IBM Software Hub CLI** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=workstations-updating-software-hub-cli> [확정] 문서 갱신일: 2026-08-20

### ★우리 판정:

- ① ★현장 절차 발견 - "옛 cpd-cli 설치본의 cpd-cli-workspace 디렉토리를 새 14.3.1 설치본으로 복사하라"(Important) + work 하위 디렉토리 전원 rwx(0777) 확인.  
→ 모비스 현장에서 14.1.3→14.3.1 전환 시 해야 할 일. 단 우리는 CPD\_CLI\_MANAGE\_WORKSPACE 로 워크스페이스를 명시 지정하므로 복사 대신 변수 지정으로 갈음 가능 [우리 방식 유지]
- ② "옛 설치본 삭제" 권고 - △모비스는 백업(5.1.3 국면)에 14.1.3 이 다시 필요하므로 삭제하지 않는다. 문서는 단일 교체 시나리오 - 우리 버전별 폴더 전략(cli-v14.x.x/)이 안전한 이유
- ③ CPD\_CLI\_MANAGE\_WORKSPACE 공식 정의 확보 [확정] - 기본값 없음, 실행 위치 기준 생성, 변수로 고정 가능. 우리 3-down.sh·1번 스크립트 방식의 정보 근거
- ④ OLM\_UTILS\_LAUNCH\_ARGS - 프록시로 미러/CASE 받을 땐 CA 마운트 필수(Important). env.sh 주석에 이미 반영돼 있음 ✓. K8s 인증서 마운트 변형도 있음
- ⑤ 기본은 olm-utils-v4 를 Entitled Registry 에서 자동 pull - OLM\_UTILS\_IMAGE 재정의는 프리미엄 라이선스(olm-utils-premium-v4) 경우. 모비스 = 표준 → 재정의는 폐쇄망 주 소용만
- ⑥ 확인 명령: cpd-cli manage restart-container (버전 확인 겸)

## Procedure (요지 + 명령)

1. GitHub IBM/cpd-cli 에서 14.3.1 다운로드 — OS·아키텍처·에디션별 표 (모비스: Linux x86\_64 EE = cpd-cli-linux-EE-14.3.1.tgz ✓확보됨)
2. 원하는 실행 위치에 풀기
3. ★옛 설치본의 cpd-cli-workspace 를 새 설치본 옆으로 복사 (Important: cpd-cli 실행파일과 같은 디렉토리에) + work 권한 확인:

```
ls -l          # drwxrwxrwx 아니면:
chmod 0777 ./work
```

4. 옛 설치본 삭제 (△모비스: 14.1.3 은 백업용으로 유지 — 판정 ②)
5. (Mac 전용 신뢰 설정 — 해당 없음)
6. Best practice: PATH 등록 (~/.bashrc 에 export PATH=<경로>:\$PATH)
7. Best practice: 환경변수 검토

```
# 워크스페이스 고정 (기본값 없음 - 실행 위치 기준 생성)
export CPD_CLI_MANAGE_WORKSPACE=<fully-qualified-directory>
# 프록시/자체서명 CA (미러·CASE 다운로드에 프록시 쓰면 필수)
export OLM_UTILS_LAUNCH_ARGS=" -v /etc/pki/ca-trust:/etc/pki/ca-trust"
# K8s API 인증서: " -v <k8-loc>:/etc/k8scert --env
K8S_AUTH_SSL_CA_CERT=/etc/k8scert"
```

Important: 설정했으면 환경변수 스크립트에도 추가 8. 확인:

```
cpd-cli manage restart-container
```

## 【파일】 1-클라이언트/3-oc갱신.md

# 원문 — Updating the OpenShift CLI (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Updating client workstations → **Updating the OpenShift CLI** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=workstations-updating-openshift-cli> [확정] 문서 갱신  
일: 2026-08-26

### ★우리 판정:

- ① 요구사항은 "클러스터와 호환되는(compatible) oc 버전" - 특정 버전 강제 없음.  
Red Hat 문서 링크만 제공 (4.14~4.22 판별). 우리 결정(4.20.24 클라이언트 하나로 4.18~4.20 커버 - 상위호환)과 충돌 없음 [유지]
- ② 시점: "OCP 클러스터를 업그레이드했다면 실행" - OCP 4.19/4.20 올라간 뒤 oc 클라이언트도 점검하라는 뜻. 우리는 처음부터 4.20.24 라 재작업 없음
- ③ Managed OpenShift(ROSA·ARO 등) 표 - 모비스 self-managed → 해당 없음

## 본문 요지

Self-managed: Red Hat "Getting started with the OpenShift CLI" 문서로 설치 (버전 4.14/4.16/4.17/4.18/4.19/4.20/4.22 링크). Managed(IBM Cloud Satellite·ROSA·ARO 등): 각 환경 문서 — 해당 없음.

## 【파일】 1-클라이언트/4-helm설치.md

# 원문 — Installing the Helm CLI (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Updating client workstations → **Installing the Helm CLI** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=workstations-installing-helm-cli> [확정] 문서 갱신일: 2026-07-15

## ★우리 판정:

- ① "선택이지만 권장" 확정 – 런북 p.13 "(선택)" 표기와 일치. 우리 tools 에 3.19.0 확보 ✓
- ② 용도가 구체적으로 명시됨 [수확]:
  - 클러스터 관리자: `case-download(--cluster_resources).apply-scheduler.setup-control-center` 디버깅 ← ★`--cluster_resources` 가 여기 등장 – p.50 전역 리소스 작업에서 문제 나면 helm 으로 디버깅한다는 것
  - 인스턴스 관리자: `install-components.uninstall-components` 디버깅
- ③ 없으면 olm-utils 컨테이너 안에 exec 해서 디버깅해야 – helm 지참이 편한 이유

## 본문 요지

Install the Helm CLI (optional but recommended) — Helm 에 의존하는 cpd-cli 명령의 이슈 디버깅을 쉽게 한다. 설치하는 Helm 공식 문서. 없으면 olm-utils-v4 컨테이너에 exec 해서 디버깅해야 한다.

## 【파일】 2-폐쇄망준비/1-허브.md

# 원문 — Preparing to run upgrade commands in a restricted network (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → **Preparing to run upgrades in a restricted network** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-07-15

## ★우리 판정:

- ① 절의 성격: "폐쇄망으로 들어가기 전, 인터넷 연결 상태에서 워크스테이션에 챙겨둘 것" = olm-utils tar + CASE 패키지. 우리 1번 스크립트가 이미 한 일의 정보
- ② 스킵 조건: 워크스테이션이 항상 인터넷 연결이면 이 절 통째 스킵 – 모비스는 폐쇄망 → 해당 (스킵 불가)
- ③ "한 번 하고 파일을 각 워크스테이션에 전송해도 됨" – 우리 반출 방식과 일치

## 본문 요지

폐쇄망에서 업그레이드 명령을 돌리려면, 방화벽 안으로 옮기기 전에 클라이언트 워크스테이션을 준비한다. 전제: 준비  
시점엔 인터넷 연결. 하위 2건: ①olm-utils-v4 이미지 확보 ②CASE 패키지 다운로드.

=====

【파일】 2-폐쇄망준비/2-olm-utils확보.md

=====

# 원문 — Obtaining the olm-utils-v4 image ... in a restricted network (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to run upgrades in a restricted network → **Obtaining the olm-utils-v4 image** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-08-20

★우리 판정:

- ① 시나리오 두 갈래: (a)같은 워크스테이션을 안으로 반입 → restart-container 로 미리 pull  
(b)★다른 워크스테이션 — 모비스 방식: save-image → tar 전송 → load-image → OLM\_UTILS\_IMAGE
- ② ★접미 확정 [5.3.x 공식]: save-image·load-image·OLM\_UTILS\_IMAGE 전부  
\${VERSION}.amd64 접미 포함. 런북 p.36 의 접미 없는 :\$VERSION 은 구식/부정확 표기  
→ 4-site-push 스크립트 olm 단계 접미 반영 [9/2 수정]
- ③ ★문서 자체의 파일명 오락가락 발견: amd64 절만 "icr.io\_cpd\_olm-utils-v4\_..." 라 쓰고  
ppc64le/s390x 절은 "icr.io\_cpopen\_cpd\_..." — 실물은 cpopen 포함(9/2 실측 일치).  
보정목록 D(파일명 표기 차이)의 근원이 IBM 문서 자체 오타였음을 확인
- ④ 검증 문구 확보: load 성공 메시지 = "Loaded image: icr.io/cpopen/cpd/olm-utils-v4:  
\${VERSION}.amd64" (런북의 "olm-utils:latest" 메시지와 다름 — 5.3.x 기준이 정확)
- ⑤ 프리미엄(olm-utils-premium-v4) 분기는 모비스 표준 라이선스 → 해당 없음

## 본문 요지 + 명령 (모비스 해당 = 다른 워크스테이션 방식, x86-64)

인터넷 워크스테이션에서:

```
cpd-cli manage save-image \  
--from=icr.io/cpopen/cpd/olm-utils-v4:${VERSION}.amd64  
# → work/offline 에 tar.gz 저장 (실물명 icr.io_cpopen_cpd_olm-utils-v4_...)
```

tar 를 클러스터 쪽 워크스테이션의 work/offline 으로 전송 후:

```
cpd-cli manage load-image \
--source-image=icr.io/cpopen/cpd/olm-utils-v4:${VERSION}.amd64
# 성공: Loaded image: icr.io/cpopen/cpd/olm-utils-v4:${VERSION}.amd64
export OLM_UTILS_IMAGE=icr.io/cpopen/cpd/olm-utils-v4:${VERSION}.amd64
```

(같은 워크스테이션 반입 방식이면: 인터넷 연결 중 restart-container 로 pull 해두면 끝 — SUCCESS 3줄 확인)

=====

## 【파일】 2-폐쇄망준비/3-CASE다운로드.md

=====

# 원문 — Downloading CASE packages ... in a restricted network (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to run upgrades in a restricted network →

**Downloading CASE packages** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=pruirm-downloading-case-packages> [확정] 문서 갱신일: 2026-07-15

### ★우리 판정:

- ① ★워크스페이스 이관 시 필수 3단계 발견 (Important) – CASE 를 다른 워크스테이션으로 옮기면: `chown -R 1001 ./work → chmod -R 775 ./work → restart-container`.  
모비스 = 워크스페이스 통째 반입이므로 ★현장 반입 직후 이 3단계 필수  
→ 4-site-push 스크립트 olm 단계에 반영 [9/2 수정]  
(1001 = olm-utils 컨테이너 내부 사용자 – 권한 안 맞으면 컨테이너가 work 를 못 읽음)
- ② CASE 다운로드 명령 = 우리 1번 스크립트 case 단계와 동일 (--patch\_id 포함, GitHub/OCI 두 갈래)
- ③ CASE 가 필요해지는 후속 작업 목록 명시: 사설 레지스트리 준비·전제소프트웨어·공유 컴포넌트·SWH 업그레이드 – 전부 워크스페이스의 CASE 를 전제  
→ p.50 전역 리소스도 이 CASE 재사용 가능성 ↑ [④-b 문서로 확정 예정]

## 본문 요지 + 명령

다운로드 (인터넷 워크스테이션 — 환경변수 source 후):

```
# GitHub
cpd-cli manage case-download \
--components=${COMPONENTS} \
--release=${VERSION} \
--patch_id=${PATCH_ID}
# OCI (icr.io): 위에 --from_oci=true 추가
```

결과: work 디렉토리에 CASE 저장.

★Important — CASE 를 다른 워크스테이션으로 이관하면 각 워크스테이션에서:

```
# work 를 품은 디렉토리에서
chown -R 1001 ./work
chmod -R 775 ./work
cpd-cli manage restart-container
```

## 【파일】 3-사설레지스트리/1-허브.md

# 원문 — Preparing to run upgrades from a private container registry (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → **Preparing to run upgrades from a private container registry**

URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-03-11

### ★우리 판정:

- ① 허브 – 하위 2건: 미러링 / olm-utils 를 사설에서 pull 하도록 전환.  
우리 1번 미러(완료)·1-site-push 스크립트 push 단계의 정보 자리
- ② 스킵 조건: Entitled Registry 에서 직접 pull 하는 환경만 스킵 –  
모비스 = 사설 레지스트리 → 필수
- ③ 담당 구분: 운영팀 + 레지스트리 관리자 협업 명시

## 본문 요지

사설 레지스트리를 쓰면 5.3 이미지를 Entitled Registry 에서 사설로 미러해야 한다. 하위 2건:

1. Mirroring images to a private container registry — 5.1 을 사설로 미러했었다면 5.3 이미지도 업그레이드 전에 미러
2. Pulling the olm-utils-v4 image from the private registry — 사설에 olm-utils 가 올라갔으면 cpd-cli 가 거기서 pull 하도록 갱신

## 【파일】 3-사설레지스트리/2-미러링-허브.md

# 원문 — Mirroring images to a private container registry (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to run upgrades from a private registry → **Mirroring images to a private container registry** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=prufpcr-mirroring-images-private-container-registry> [확정] 문서 갱신일: 2026-08-26

★우리 판정:

- ① 미러 방식 2갈래: (a)직접(인터넷+사설 동시 연결 워크스테이션) / (b)★중간 레지스트리(폐쇄망) – 모비스 = (b). 하위 페이지도 2건으로 갈라짐
- ② ★중간 레지스트리 운반 3옵션 공식 명시: 노트북 반입 / USB / scp·sftp 전송. 모비스 = 세 번째(전송) – "인터넷 워크스테이션(100.86)에서 중간 레지스트리에 미러 → 전송 → 사설 연결 워크스테이션에서 사설로 미러" = 우리 반출 계획 그대로 [정본 확인]
- ③ "mirror-images 가 중간 레지스트리를 워크스테이션에 자동 구성" – 127.0.0.1:12443 자동 기동의 공식 근거 (1번 스크립트·1-site-push 의 전제)
- ④ 다중 인스턴스면 컴포넌트 합집합으로 한 번에 미러 가능 – 모비스 단일이라 무관
- ⑤ 전제: Private container registry requirements 충족 (링크 – 필요시 수집)

## 본문 요지

방식 선택 질문: "인터넷과 사설 레지스트리에 동시에 연결되는 워크스테이션을 둘 수 있나?"

- Yes → 직접 미러
- No(폐쇄망) → 중간 레지스트리 경유. mirror-images 명령이 중간 레지스트리를 자동 구성. 방화벽 안으로 옮기는 방법 3가지: ①노트북(같은 장비로 양쪽 작업) ②USB(워크스테이션 2대) ③scp/sftp 전송(워크스테이션 2대) ← 모비스 방식

하위 2건: 직접 미러 / ★중간 레지스트리 미러 (다음 수집 대상)

=====

【파일】 3-사설레지스트리/3-olm-utils-pull.md

=====

# 원문 — Pulling the olm-utils-v4 image from the private registry (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to run upgrades from a private registry → **Pulling the olm-utils-v4 image** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=prufpcr-pulling-olm-utils-v4-image-from-private-container-registry> [확정] 문서 갱신일: 2026-08-20

★우리 판정:

- ① ★폐쇄망 최종형 확정 [4-환경변수.md 의 대조 대기 해소]:  
`export OLM_UTILS_IMAGE=${PRIVATE_REGISTRY_LOCATION}/cpopen/cpd/olm-utils-v4:${VERSION}.amd64`  
 - 사설 주소에서도 아키 접미(.amd64) 포함이 정식.  
 런북 p.48(접미 없음)은 부정확 → env.sh 주석 갱신 [9/2]
- ② 시점: 사설로 미리 완료 후 + manage 명령 실행 전, 워크스테이션마다 1회
- ③ 이후 manage 명령은 사설에서 olm-utils 를 자동 pull - 별도 load 불필요  
 (tar load 방식은 2-폐쇄망준비 문서의 경로 - 사설에 올라가 있으면 이 방식이 더 단순)
- ④ 프리미엄 변형(경로 /cp/cpd/) - 모비스 해당 없음

## 본문 요지 + 명령 (x86-64)

전제: 사설 미리 완료 + cpd-cli 설치. 환경변수 스크립트의 OLM\_UTILS\_IMAGE 를 사설 주소로 (없으면 추가):

```
export OLM_UTILS_IMAGE=${PRIVATE_REGISTRY_LOCATION}/cpopen/cpd/olm-utils-v4:${VERSION}.amd64
```

결과: manage 명령 실행 시 사설 레지스트리에서 자동 pull.

=====

## 【파일】 4-클러스터준비/1-허브.md

=====

## 원문 — Preparing your cluster for a new version (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → **Preparing your cluster** URL: [미확인 — 주소 받으면 기입] 문서 갱신  
 일: 2026-03-11

### ★우리 판정:

- ① ★구조 발견: 이 절의 전역리소스(④-b)·폴시크릿(④-c)은 "스케줄러가 설치된 경우" 조건부 - 스케줄러용 클러스터롤·바인딩·시크릿 얘기다.  
 → 런북 p.50 의 case-download --cluster\_resources 는 이쪽이 아니라  
 ⑤-인스턴스준비의 "Updating cluster-scoped resources for the instance" 가 원본일 가능성 [⑤ 수집으로 확정 예정 - 폐쇄망 분기의 답도 거기]
- ② 이 절 필수분: ④-a 전제소프트웨어(서비스별) + ④-d 공유 컴포넌트 (License Service + 스케줄러) 업그레이드
- ③ 클러스터당 1회 (One-time setup)

## 본문 요지

업그레이드 전에 전제 소프트웨어와 공유 클러스터 컴포넌트를 갱신한다. 클러스터 관리자 담당, 클러스터당 1회. 하위 4건:

1. Upgrading prerequisite software — 여러 서비스의 전제 SW 업그레이드
2. Updating cluster-scoped resources for shared cluster components — ★스케줄러 설치 시: 스케줄러용 클러스터롤·바인딩 갱신
3. Creating image pull secrets for shared cluster components — ★스케줄러 설치 시: 스케줄러 설치 프로젝트에 풀 시크릿
4. Upgrading shared cluster components — License Service + 스케줄러 업그레이드

=====

## 【파일】 4-클러스터준비/2-전제소프트웨어.md

=====

# 원문 — Upgrading prerequisite software (5.1→5.3)

## [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing your cluster → **Upgrading prerequisite software** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-07-15

★우리 판정 (모비스 서비스 = ws-ws\_runtimes-wml-openscale-ws\_pipelines-db2oltp):

- ① 전제 SW 5종 표에서 모비스 탑재 해당은 GPU 조건부 2종뿐:
  - NFD Operator·NVIDIA GPU Operator - WML\*·WS Runtimes\* 가 목록에 있으나 \* = "GPU 쓰는 상황만". → 실측: 모비스 클러스터에 GPU 노드·해당 오퍼레이터 설치 여부 확인. 없으면 ④-a 통째 해당 없음
- ② 해당 없음 확정 3종: RHOAI(모비스 서비스 중 의존 없음 - wml 은 RHOAI 목록에 없음) / MCG(Discovery·Speech·Assistant·Orchestrate 용) / Serverless Knative(同)
- ③ ★트리거 조건 수확: NFD 는 "OCP 업그레이드 + installPlanApproval: Manual" 때 갱신
  - 필요 - OCP 4.19/4.20 올린 뒤 점검 항목. 우리 imageset 에 nfd 가 든 이유와 부합
- ④ △imageset 재검토 후보: 우리 4.19/4.20 imageset 의 rhods-operator(RHOAI)는 ②에 따르면 모비스 불필요 가능성 - 용량 절약 여지 [사수 확인 후보]

## 본문 요지 (표)

전제 SW ↔ 필요 서비스 ↔ 갱신 조건:

1. NFD Operator — GPU 계열 서비스(\*) — OCP 업그레이드 + Manual 승인 시
2. NVIDIA GPU Operator — GPU 계열(\*) — 오퍼레이터 지원 종료 시
3. RHOAI — IKC·Speech·wx.ai 계열 — SWH 새 버전이 새 RHOAI 요구 시
4. MCG — Discovery·Speech·Assistant·Orchestrate — OCP 업그레이드 시
5. Serverless Knative — Assistant·Orchestrate — Events Operator 신판 등

하위 페이지 3건(GPU 오퍼레이터/RHOAI/MCG) — 모비스 해당 없음 판정이라 수집 생략 (GPU 실측 결과 해당 시 GPU 페이지만 추가 수집)

=====

【파일】 4-클러스터준비/3-전역리소스-공유.md

=====

# 원문 — Updating cluster-scoped resources for shared cluster components (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing your cluster → **Updating cluster-scoped resources for shared cluster components** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=cluster-updating-scoped-resources-shared-components> [확정] 문서 갱신일: 2026-07-15

★우리 판정:

- ① ★★폐쇄망 퍼즐 해소 — 명령 변형에 "The CASE packages are already on the client workstation" 이 공식으로 존재! 명령 자체는 GitHub 판과 동일 — 즉 CASE 가 워크스페이스에 이미 있으면 case-download --cluster\_resources 는 다운로드 없이 로컬 CASE 로 yaml 만 생성한다 [문서 확정].  
→ 런북 p.50 을 현장에서 그대로 쳐도 됨 (워크스페이스 통째 반입 전제 — 우리 계획이 바로 그것). C13 등록 불필요로 격하, 100.86 사전 생성은 보험 선택지로만
- ② 이 페이지는 스케줄러판: --components=scheduler + --scheduler\_ns 추가. 스케줄러 설치 시에만 해당 [실측 후 결정]. 인스턴스판(런북 p.50 원본)은  
⑤-인스턴스준비에서 재확인 예정 — 같은 패턴일 것 [잠정]
- ③ 생성물 정의 확보: cluster\_scoped\_resources.yaml = CRD·클러스터를. 클러스터롤바인딩·웹훅 (스케줄러용)
- ④ 보관 팁: mv 로 \${VERSION}-\${NS}-cluster\_scoped\_resources.yaml 개명 — 기록 습관

## 본문 요지 + 명령

전제: 환경변수 source. CASE 가 워크스테이션에 있어야 함.

1. 스케줄러용 전역 리소스 yaml 생성 (CASE 로컬 존재 시 — 폐쇄망 OK):

```
cpd-cli manage case-download \  
--components=scheduler \  
--release=${VERSION} \  
--patch_id=${PATCH_ID} \  
--scheduler_ns=${PROJECT_SCHEDULING_SERVICE} \  
--cluster_resources=true
```

(GitHub/OCI/자체 OCI 저장소 변형 동일 구조) 2. work 디렉토리로 이동 (기본 cpd-cli-workspace/olm-utils-workspace/work) 3. \${OC\_LOGIN} 4. 적용:

```
oc apply -f cluster_scoped_resources.yaml \
--server-side \
--force-conflicts
```

5. (선택) 기록용 개명:

```
mv cluster_scoped_resources.yaml ${VERSION}-${PROJECT_SCHEDULING_SERVICE}-
cluster_scoped_resources.yaml
```

=====

## 【파일】 4-클러스터준비/4-폴시크릿-공유.md

=====

# 원문 — Creating image pull secrets for shared cluster components (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing your cluster → **Creating image pull secrets for shared cluster components** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=pyc-creating-image-pull-secrets-shared-cluster-components> [확정] 문서 갱신일: 2026-07-15

### ★우리 판정:

- ① 스케줄러 조건부 – 설치 시 스케줄러 NS 에 풀 시크릿 생성 (스케줄러 업그레이드 전).
- ② ★런북 누락 후보: 런북 p.52 는 오퍼레이터.오퍼랜드 두 NS 에만 시크릿을 만들 – 스케줄러 NS 용 시크릿 절이 런북에 없음. 모비스에 스케줄러가 설치돼 있다면 (실측 대기) 이 절도 실행 목록에 추가해야 함 [스케줄러 실측과 한 묶음]
- ③ dockerconfig.json 형식은 런북 p.52 와 동일 패턴 (사설: PRIVATE\_REGISTRY\_LOCATION + IMAGE\_PULL\_CREDENTIALS). Entitled 형은 cp.icr.io + icr.io 두 auth – 참고

## 본문 요지 + 명령 (사설 레지스트리)

```
${OC_LOGIN}
cat <<EOF > dockerconfig.json
{
  "auths": {
    "${PRIVATE_REGISTRY_LOCATION}": {
      "auth": "${IMAGE_PULL_CREDENTIALS}"
```

```

    }
  }
}
EOF
oc create secret docker-registry ${IMAGE_PULL_SECRET} \
--from-file ".dockerconfigjson=dockerconfig.json" \
--namespace=${PROJECT_SCHEDULING_SERVICE}

```

## 【파일】 4-클러스터준비/5-공유컴포넌트.md

# 원문 — Upgrading shared cluster components (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing your cluster → **Upgrading shared cluster components**

URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=cluster-upgrading-shared-components> [확정]

문서 갱신일: 2026-08-20

### ★우리 판정:

- ① License Service 업그레이드(apply-cluster-components)는 런북 p.48~49 와 동일  
[대조 일치]  
+ 추가 수확: "[SUCCESS] The apply-cluster-components command ran successfully"  
메시지를 기다렸다가 다음으로 (런북에 없던 대기 조건)
- ② ★스케줄러 업그레이드 명령 확보 – 런북에 없음! (런북은 설치 확인만 하고 업그레이드 안 함):  

```

apply-scheduler --release --patch_id --license_acceptance --scheduler_ns
--image_pull_prefix --image_pull_secret
→ 스케줄러 조건부 누락 3종 세트 완성: ④-b 전역리소스 + ④-c NS 풀시크릿 + ④-d
apply-scheduler

```

모비스 스케줄러 실측 = 이 3종의 실행 여부를 한 번에 결정
- ③ 다중 인스턴스면 공유 컴포넌트는 최신 목표 버전으로 (모비스 단일 – 무관)
- ④ cert-manager 감지 언급은 이 페이지에 없음 – 그 설명은 런북 서두(p.6)와  
Migrating to RH cert-manager 문서 쪽 [기존 이해 유지]

## 본문 요지 + 명령

사전 확인 (런북과 동일):

```

oc get deployment -A | grep ibm-licensing-operator # License NS 확인
oc get scheduling -A                               # 스케줄러 설치 확인

```

1. \${CPDM\_OC\_LOGIN}
2. License Service 업그레이드:

```
cpd-cli manage apply-cluster-components \
--release=${VERSION} \
--patch_id=${PATCH_ID} \
--license_acceptance=true \
--licensing_ns=${PROJECT_LICENSE_SERVICE}
```

(PROJECT\_CS\_CONTROL 변형 동일 구조) ★"[SUCCESS] ... ran successfully" 나올 때까지 대기 후 진행 3. License 파드 Running/Completed 확인 4. ★스케줄러 설치 시 — 업그레이드 (런북 누락분):

```
cpd-cli manage apply-scheduler \
--release=${VERSION} \
--patch_id=${PATCH_ID} \
--license_acceptance=true \
--scheduler_ns=${PROJECT_SCHEDULING_SERVICE} \
--image_pull_prefix=${IMAGE_PULL_PREFIX} \
--image_pull_secret=${IMAGE_PULL_SECRET}
```

5. 스케줄러 파드 Running/Completed 확인

## 【파일】 5-인스턴스준비/1-허브.md

# 원문 — Preparing to upgrade an instance (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → **Preparing to upgrade an instance of IBM Software Hub** URL:  
[미확인 — 주소 받으면 기입] 문서 갱신일: 2026-07-15

### ★우리 판정:

- ① ★Events Operator 절 = "wx Assistant 또는 wx Orchestrate 포함 인스턴스만" — 모비스는 둘 다 미설치 → 해당 없음 확정. 런북 누락 후보 ⑥ 해소 (런북이 빠뜨린 게 아니라 모비스 무관이라 안 실은 것으로 정리)
- ② 전역리소스 인스턴스판("for the platform and services") — 런북 p.50 의 원본 확정 대상. 다음 수집에서 폐쇄망 CASE 재사용 변형 재확인
- ③ NSS 재인증·사용자 재인증 — 둘 다 "설치 때 minimum RBAC 를 줬다면" 조건부. 런북 p.52 의 jq 확인(true 면 통과)과 일치하는 구조
- ④ entitlements — 조건 없는 서술("You are required to keep a record...") —

런북 Stage1 누락 후보 ⑨ 유지, 본문 보고 최종 판정  
⑤ 인스턴스마다 반복 - 모비스 단일 1회

## 본문 요지 (하위 6건)

1. Checking the health of your cluster — health 명령 3종 (런북 p.50)
2. Updating the cluster-scoped resources for the platform and services — 인스턴스 컴포넌트용 클러스터롤-바인딩 생성 (런북 p.50~52 원본)
3. Upgrading the IBM Events Operator — ★wxA/wxO 포함 시만 → 모비스 해당 없음
4. Reauthorizing the NamespaceScope operator (min RBAC 준 경우)
5. Reauthorizing an instance administrator (min RBAC 준 경우)
6. Applying your entitlements — License Service 계량용 엔타이틀먼트 적용

=====

【파일】 5-인스턴스준비/2-건강검진.md

=====

## 원문 — Checking the health of your cluster (5.1 → 5.3)

[2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to upgrade an instance → **Checking the health of your cluster** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-08-20

### ★우리 판정:

- ① ★런북(p.50)은 3종인데 공식은 4종 - 네 번째 검사 확보:  
cpd-cli health network-connectivity --  
control\_plane\_ns=\${PROJECT\_CPD\_INST\_OPERANDS}  
(오퍼레이터↔오퍼랜드 파드 연결까지 검사) → 실행판 점검에 추가 가치
- ② ★폐쇄망 함정 주석 확보: cluster 검사의 "Image Registry Connectivity Test" 는  
icr.io.github.mirror.openshift.com 도달 검사라 폐쇄망에선 FAIL 이 정상 -  
"If you pull images from a private container registry, [FAIL] is an  
acceptable  
result" [원문]. 현장에서 FAIL 보고 당황하지 않기
- ③ 합격 기준 숫자 확보: 노드 자원 사용 90% 미만 / 네트워크 처리량 최소 350 MB/s
- ④ 성격: 선택이지만 강력 권장(strongly recommended)

## 본문 요지 + 명령

`${CPDM_OC_LOGIN}`

# ① 클러스터 (버전·MCP·CO·etcd - ★폐쇄망은 레지스트리 도달검사 FAIL 정상)  
cpd-cli health cluster

```
# ② 노드 (READY + 자원 90% 미만)
cpd-cli health nodes

# ③ 네트워크 처리량 (최소 350 MB/s - 사설 레지스트리 변형 있음)
cpd-cli health network-performance \
--image-prefix=${PRIVATE_REGISTRY_LOCATION}

# ④ ★네트워크 연결 (런북에 없는 검사 - 오퍼레이터↔오퍼랜드 연결 포함)
cpd-cli health network-connectivity \
--control_plane_ns=${PROJECT_CPD_INST_OPERANDS}
```

기대 결과 표 요지: Cluster Version-MCP-Cluster Operator-etcd = SUCCESS / CSR = SKIP 가능 / 폐쇄망 레지스트리 도달 = FAIL 허용 / 노드 READY·자원<90% / throughput ≥350MB/s / 연결 4종 SUCCESS

=====

## 【파일】 5-인스턴스준비/3-전역리소스-인스턴스.md

=====

# 원문 — Updating the cluster-scoped resources for the platform and services (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to upgrade an instance → **Updating the cluster-scoped resources for the platform and services** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-07-15

### ★우리 판정:

- ① ★런북 p.50~52 의 원본 확정 - 절차 5단계(생성→work 이동→로그인→apply --server-side --force-conflicts→mv 개명)가 런북과 정확히 일치
- ② ★폐쇄망 퍼즐 최종 종결 - 인스턴스판에도 명령 변형 1순위가 "The CASE packages are already on the client workstation". 워크스페이스(CASE 포함) 반입 전제로 런북 p.50 현장 실행 가능 [문서 확정]
- ③ Restriction 수확: Helm 기반 설치를 지원하는 컴포넌트만 yaml 생성 - 비Helm 컴포넌트는 생성물 없음(빈 결과가 에러 아님을 이해하는 근거)
- ④ 생성물 = 컴포넌트별 CRD·클러스터롤·클러스터롤바인딩·(해당 시)웹훅. CRD 는 클러스터당 1회, 롤바인딩은 인스턴스별 적용 구분 설명
- ⑤ 성격: required (필수) / 인스턴스별 반복

## 본문 요지 + 명령

전제: 환경변수 source. CASE 가 워크스테이션에 있어야 함.

1. cluster\_scoped\_resources.yaml 생성 — 변형 4종: ★CASE 로컬 존재(폐쇄망) / GitHub / IBM OCI / 자체 OCI (런북 p.50 판 명령 = 이 절차의 구현:

```
cpd-cli manage case-download \
--components=${COMPONENTS} \
--release=${VERSION} \
--operator_ns=${PROJECT_CPD_INST_OPERATORS} \
--cluster_resources=true
```

) 2. work 디렉토리로 이동 (기본 cpd-cli-workspace/olm-utils-workspace/work) 3. \${OC\_LOGIN} 4. 적용:

```
oc apply -f cluster_scoped_resources.yaml \
--server-side \
--force-conflicts
```

5. (선택) 기록용 개명:

```
mv cluster_scoped_resources.yaml ${VERSION}-${PROJECT_CPD_INST_OPERATORS}-
cluster_scoped_resources.yaml
```

=====

【파일】 5-인스턴스준비/3b-Events오퍼레이터.md

=====

## 원문 — Upgrading the IBM Events Operator for wxA/wxO (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to upgrade an instance → **Upgrading the IBM Events Operator** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-07-15

★우리 판정:

- ① 모비스 해당 없음 확정 [본문 재확인] – 조건이 이중:
    - (a) 5.3.1+ 로 갈 때만 + (b) wx Assistant 또는 wx Orchestrate 설치 시만.
    - 모비스는 (b) 불충족 → 통째 건너뛴. 런북 누락 아님 최종 확정
  - ② (참고) ★삼성 랩엔 유관 – 삼성은 wxo 설치본이라, 삼성 5.4 업그레이드 시나리오에선 이 절(deploy-events-operator)이 등장할 수 있음.
- 선행조건: Serverless Knative Eventing 업그레이드

### 본문 요지 + 명령 (참고 보존 — 삼성용)

조건: 5.3.1+ 업그레이드 + wxA/wxO 설치 인스턴스. 선행: Upgrading Red Hat OpenShift Serverless Knative Eventing.

```

${CPDM_OC_LOGIN}
cpd-cli manage deploy-events-operator \
--release=${VERSION} \
--events_operator_ns=${PROJECT_CPD_INST_OPERATORS} \
--events_operand_ns=${PROJECT_CPD_INST_OPERANDS}

```

## 【파일】 5-인스턴스준비/4-NSS재인증.md

# 원문 — Reauthorizing the NamespaceScope operator (min RBAC) (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to upgrade an instance → **Reauthorizing the NamespaceScope operator** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-08-20

### ★우리 판정:

- ① jq 확인 명령 = 런북 p.52 와 동일 [일치]. true → 스킵 / false → 재인증 필요.  
(authorize-instance-topology 로 설치한 경우도 스킵)
- ② ★런북엔 "follow the doc" 링크뿐이던 false 대응 절차 전문 확보:  
show-minimum-rbac 로 role yaml 생성 → 오퍼레이터.오퍼랜드 NS 에 role apply  
→ 같은 이름 RoleBinding 생성(SA=ibm-namespace-scope-operator).  
현장에서 false 나와도 이제 막히지 않음
- ③ 테더드 변형 존재 – 모비스 해당 없음
- ④ 모비스 실측: jq 명령 결과 true/false 가 이 절 실행 여부를 결정

## 본문 요지 + 명령

판별 (런북 p.52 와 동일):

```

oc get role nss-managed-role-from-${PROJECT_CPD_INST_OPERATORS} \
-n ${PROJECT_CPD_INST_OPERATORS} \
-o json | jq 'any(.rules[.].apiGroups[.]; . == "*")'
# true = 재인증 불필요 / false = 아래 절차 실행

```

false 일 때 (★런북에 없던 실제 절차):

```

${CPDM_OC_LOGIN}
# ① 최소 RBAC role yaml 생성 (work 디렉토리에 생성됨)
cpd-cli manage show-minimum-rbac \

```

```
--cpd_operator_ns=${PROJECT_CPD_INST_OPERATORS} \
--components=${COMPONENTS} \
--release=${VERSION} \
--patch_id=${PATCH_ID}
# ② role 을 두 NS 에 적용
oc apply -f nss-managed-role-from-${PROJECT_CPD_INST_OPERATORS}.yaml --
namespace=${PROJECT_CPD_INST_OPERATORS}
oc apply -f nss-managed-role-from-${PROJECT_CPD_INST_OPERATORS}.yaml --
namespace=${PROJECT_CPD_INST_OPERANDS}
# ③ RoleBinding 생성 - 두 NS 각각 (heredoc, SA=ibm-namespace-scope-operator,
#   roleRef=위 role. 전문은 원문 - 테더드 변형은 모비스 무관)
```

## 【파일】 5-인스턴스준비/5-사용자재인증.md

# 원문 — Reauthorizing an instance administrator (min RBAC) (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to upgrade an instance → **Reauthorizing an instance administrator with the minimum RBAC** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-08-20

### ★우리 판정:

- ① 스킵 조건: "프로젝트에 admin 룰을 준 사용자면 스킵" - 모비스는 클러스터 관리자가 직접 업그레이드하는 구도로 추정 → 해당 없음 가능성 높음  
[잠정 - 현장 운영 방식(누가 업그레이드 계정인지) 실측]
- ② 해당 시 절차 전문 확보 - 런북 p.52 는 링크뿐(B7)이던 것:  
show-minimum-rbac --role\_name → yaml 2종(instance-admin·instance-crs)  
→ 두 NS 에 role 4회 apply → add-role-to-user 4회
- ③ 그룹 단위 부여 팁: add-role-to-group 대체 가능
- ④ 테더드 변형 - 모비스 무관

## 본문 요지 + 명령 (해당 시)

```
${CPDM_OC_LOGIN}
export ROLE_NAME=<이 인스턴스의 min RBAC 식별자>
# ① yaml 2종 생성 (work 에)
cpd-cli manage show-minimum-rbac \
--components=${COMPONENTS} \
--release=${VERSION} \
--patch_id=${PATCH_ID} \
--role_name=${ROLE_NAME}
# 생성물: ${ROLE_NAME}-cpd-instance-admin.${VERSION}.yaml
#         ${ROLE_NAME}-cpd-instance-crs.${VERSION}.yaml
```

```
# ② role 적용 - admin.crs 각각을 오퍼레이터.오퍼랜드 NS 에 (4회)
oc apply -f ${ROLE_NAME}-cpd-instance-admin.${VERSION}.yaml --
namespace=${PROJECT_CPD_INST_OPERATORS}
oc apply -f ${ROLE_NAME}-cpd-instance-crs.${VERSION}.yaml --
namespace=${PROJECT_CPD_INST_OPERATORS}
oc apply -f ${ROLE_NAME}-cpd-instance-admin.${VERSION}.yaml --
namespace=${PROJECT_CPD_INST_OPERANDS}
oc apply -f ${ROLE_NAME}-cpd-instance-crs.${VERSION}.yaml --
namespace=${PROJECT_CPD_INST_OPERANDS}
# ③ 사용자에게 롤 부여 (4회 - 그룹이면 add-role-to-group)
export INSTANCE_ADMIN=<사용자>
oc adm policy add-role-to-user ${ROLE_NAME}-cpd-instance-admin ${INSTANCE_ADMIN} -
--namespace=${PROJECT_CPD_INST_OPERATORS} --role-
namespace=${PROJECT_CPD_INST_OPERATORS}
oc adm policy add-role-to-user ${ROLE_NAME}-cpd-instance-crs ${INSTANCE_ADMIN} -
--namespace=${PROJECT_CPD_INST_OPERATORS} --role-
namespace=${PROJECT_CPD_INST_OPERATORS}
oc adm policy add-role-to-user ${ROLE_NAME}-cpd-instance-admin ${INSTANCE_ADMIN} -
--namespace=${PROJECT_CPD_INST_OPERANDS} --role-
namespace=${PROJECT_CPD_INST_OPERANDS}
oc adm policy add-role-to-user ${ROLE_NAME}-cpd-instance-crs ${INSTANCE_ADMIN} -
--namespace=${PROJECT_CPD_INST_OPERANDS} --role-
namespace=${PROJECT_CPD_INST_OPERANDS}
```

## 【파일】 5-인스턴스준비/6-엔타이틀먼트.md

# 원문 — Applying your entitlements (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Preparing to upgrade an instance → **Applying your entitlements**  
(+하위 without node pinning) URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-08-20

### ★우리 판정:

- ① ★런북 누락 후보 ② 판정: 누락이 아니라 조건부일 가능성 높음 - required 조건이  
(a)설치 때 apply-entitlement 를 안 돌렸거나 (b)업그레이드하며 솔루션 구성을  
바꿀 때. 모비스 업그레이드는 구성 변경 없음 → (a)만 실측하면 끝:  
★실측 항목: 설치 당시 apply-entitlement 실행 여부 (License Service 등록 상태)
- ② 하위 2갈래: 노드피닝 없이(인스턴스 관리자 가능) / 노드피닝(클러스터 관리자).  
모비스 노드피닝 사용 여부 [실측 - 미사용 추정]
- ③ 명령 형태 확보 (예시로 노출된 것):  
cpd-cli manage apply-entitlement --  
cpd\_instance\_ns=\${PROJECT\_CPD\_INST\_OPERANDS}  
--entitlement=\${LICENSE\_NAME} --production=true|false  
(라이선스별 LICENSE\_NAME 상이 - 페이지에 제품별 25종 목록)

- ④ 모비스 라이선스 종류(CPD Enterprise/Standard 등)는 [미확인 - 사수 확인]  
→ LICENSE\_NAME 확정에 필요. 제거는 remove-entitlement

## 본문 요지

- License Service 가 라이선스 조건 대비 사용량을 계량 — apply-entitlement 로 "이 인스턴스에 어떤 라이선스가 적용되는지" 알려주는 작업
- required 조건: 설치 때 안 했거나 / 솔루션 구성 변경 시
- 절차: \${CPDM\_OC\_LOGIN} → 설치된 솔루션별로 apply-entitlement 반복 (production=true/false 변형) → 제거 예정 솔루션은 remove-entitlement
- 명령 골격:

```
cpd-cli manage apply-entitlement \  
--cpd_instance_ns=${PROJECT_CPD_INST_OPERANDS} \  
--entitlement=${LICENSE_NAME} \  
--production=true
```

- 라이선스 목록: CPD EE/SE-Cognos-DataStage-IKC-OpenPages-Speech- wx.ai-wx.data-wx.governance-wxO 등 25종 (모비스 = CPD 에디션 계열 추정)

## 하위 b — 노드피닝판 (9/2 수집, 요지만)

- 조건: 솔루션 변경 + 노드피닝 사용(또는 시작) — 둘 다 참일 때만. ★모비스 노드피닝 미사용 추정 → 해당 없음 [실측: 노드에 isc-entitlement 라벨 존재 여부로 판별 가능]
- 노드피닝 = 유료(chargeable) 파드를 특정 노드에 고정해 라이선스 준수를 보장하는 방식. 식별은 호스트명 또는 라벨(isc-entitlement=값), 강제 여부는 required/preferred DuringScheduling 설정으로 갈림
- 명령은 apply-entitlement 에 피닝 변수들이 추가되는 형태 (본문 명령 블록은 페이지 접힘으로 미노출 — 필요 시 재수집)

=====

### 【파일】 6-인스턴스업그레이드/2-폴시크릿-인스턴스.md

=====

## 원문 — Creating image pull secrets for an instance (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Upgrading an instance → **Creating image pull secrets for the instance** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-07-15

★우리 판정:

- ① 런북 p.52 와 완전 일치 [대조] - dockerconfig.json 생성(사설: PRIVATE\_REGISTRY\_LOCATION + IMAGE\_PULL\_CREDENTIALS) →

- 오퍼레이터.오퍼랜드 두 NS 에 시크릿 생성. required.
- ② 용도 설명 수확: "Helm 을 쓰는 컴포넌트들이 이미지 풀 자격증명에 접근하게 보장" - 시크릿의 존재 이유
  - ③ 테더드 변형 - 모비스 무관 / 담당: 인스턴스 관리자 가능
  - ※ dockerconfig.json 내용 블록은 페이지 접힘으로 미노출 -  
런북 p.52 원본 + ④-c(공유판, Entitled 형까지)로 대체 충분

## 본문 요지 + 명령

```
${OC_LOGIN}
# dockerconfig.json 생성 (사설: ${PRIVATE_REGISTRY_LOCATION} +
#   ${IMAGE_PULL_CREDENTIALS} - 형식은 런북 p.52·④-c 참조)
oc create secret docker-registry ${IMAGE_PULL_SECRET} \
--from-file ".dockerconfigjson=dockerconfig.json" \
--namespace=${PROJECT_CPD_INST_OPERATORS}
oc create secret docker-registry ${IMAGE_PULL_SECRET} \
--from-file ".dockerconfigjson=dockerconfig.json" \
--namespace=${PROJECT_CPD_INST_OPERANDS}
```

## 【파일】 6-인스턴스업그레이드/3-SWH업그레이드.md

# 원문 — Upgrading IBM Software Hub (5.1→5.3) [9/2 수집 → 9/3 전문 승격]

경로: 5.3.x → Upgrading from Version 5.1 → Upgrading an instance → **Upgrading IBM Software Hub** URL:  
<https://www.ibm.com/docs/en/software-hub/5.3.x?topic=uish-upgrading-software-hub> 문서 갱신일: 2026-08-20 ※9/2 요약 저장분을 9/3 사용자 재붙여넣기로 전문 교체 (접힌 블록 제외 — 아래 [접힘] 표기)

★우리 판정 (런북 p.53~60 의 원본 — 대조 결과):

- ① 빠대 일치 [대조]: global search 체크 → catalog-api 체크·DB 수 세기(curl+TOKEN) → 다운타임 표(6/20/60분) → precheck\_migration.sh 전문(줄까지 동일) → 판정표 → install-components cpd\_platform → RSI 패치 → get-cr-status → health operators/operands. 런북이 이 페이지의 충실한 사본이었음
- ② ★런북에 없는 것 5건 수확:
  - (a) --preview=true 팁 - 실행 전 oc 명령 미리보기(work/preview.sh 저장).  
★PROD 첫 실행 전 안전장치로 실행판 채택 가치 높음
  - (b) --run\_storage\_tests 해설 - 스토리지 최소 성능 검사. 강력 권장,  
미달 시 옵션 제거로 진행 가능하나 문제 가능성 경고 (런북은 플래그만 있음)
  - (c) ★배치 업그레이드 선택지 - cpd\_platform 만 먼저 올린 뒤,  
--components=\${COMPONENTS} 로 서비스 전체를 한 방에 올리는  
install-components 변형(Optional). 런북의 서비스 도미노(p.61~64  
하나씩 6번)를 이 한 명령으로 대체 가능 - 실행판 선택지

- (d) health operators/operands 의 기대 결과 표 전문 (SUCCESS/SKIP 기준)
- (e) [9/3 전문에서 추가 확보] 반자동 활성화 patch 명령 –  
`use_semi_auto_catalog_api_migration: true` (판정표 3행)
- ③ 서비스별 선행(wxA 소유권 이전·Ansible isvc 삭제) – 모비스 해당 없음
- ④ [9/3 확정] 스텝3 본 명령(tethered 없음 판) 공개 – 런북 p.59·체크리스트 No.110 과 일치 확인: `--components=cpd_platform + --run_storage_tests=true + --upgrade=true` (아래 전문).  
tethered 있음 판만 접힘 잔존(모비스 해당 없음)  
+ Db2 라이선스 변형(DB2SE/DB2AE) 명령도 공개  
※precheck\_migration.sh 는 export/scripts/ 에 반출 배치 [9/3]
- ⑤ 모비스 예상: couchdb PVC 30Gi + scaleConfig 미설정(=small)  
→ "ready ... as usual" = 자동 이사 예상 [체크리스트 No.69 일치]

## 원문 전문

Upgrading IBM Software Hub (Upgrading from Version 5.1 to Version 5.3) Last Updated: 2026-08-20

To upgrade an instance of IBM Software Hub, you must grade the required operators and custom resources for the instance.

Upgrade phase:

- You are not here. Collecting required information
- You are not here. Updating your client workstation
- You are not here. Preparing to run an upgrade in a restricted network
- You are not here. Preparing to run an upgrade from a private container registry
- You are not here. Upgrading prerequisite software
- You are not here. Upgrading shared cluster components
- You are not here. Preparing to upgrade an instance
- **You are here icon. Upgrading an instance**

Who needs to complete this task?

- Instance administrator — An instance administrator can complete this task.

When do you need to complete this task?

- Repeat as needed — If you have multiple instances of IBM Software Hub on the cluster, complete this task for each instance that you want to upgrade.

Before you begin

Best practice: You can run the commands in this task exactly as written using the installation environment variables. Ensure that you added the new environment variables from Updating your environment variables script.

In addition, ensure that you source the environment variables before you run the commands in this task.

### Service-specific prerequisite tasks

If you have any of the following services on this instance of IBM Software Hub, complete the required steps before you upgrade IBM Software Hub (둘 다 모비스 해당 없음):

- watsonx Assistant: "Transfer ownership of the ..." [일부만 노출 — 원문 문장 잘림]
- watsonx Code Assistant for Red Hat Ansible Lightspeed: Delete the ibm-granite-3b-code-v1 and ibm-granite-20b-code-8k-ansible InferenceService object:

```
oc delete isvc ibm-granite-3b-code-v1 ibm-granite-20b-code-8k-ansible \
-n ${PROJECT_CPD_INST_OPERANDS} \
--ignore-not-found
```

## Common core services

Before you upgrade IBM Software Hub, check for the whether the following pods are running in this instance of IBM Software Hub:

a. Check whether the global search pods are running:

```
oc get pods --namespace=${PROJECT_CPD_INST_OPERANDS} | grep elasticsea-0ac3
```

- If the command returns an empty response, proceed to the next step.
- If the command returns a list of pods, review "Upgrades fail when global search is configured incorrectly" to determine whether you have any configurations that could cause issues during upgrade.

b. Check whether the catalog-api pods are running:

```
oc get pods --namespace=${PROJECT_CPD_INST_OPERANDS} | grep catalog-api
```

- If the command returns an empty response, you are ready to upgrade IBM Software Hub.
- If the command returns a list of pods, review the following guidance to determine how long the catalog-api service will be down during upgrade.

When you upgrade the common core services to IBM Software Hub Version 5.3, the underlying storage for the catalog-api service is migrated to PostgreSQL.

During the final stages of the migration, the catalog-api service is offline, and services that are dependent on the service are not available. The duration of the migration depends on the number of assets and relationships that are stored in the instance. The duration of the outage depends on the number of databases (projects, catalogs, and spaces) in the instance. In a typical upgrade scenario, the outage should be significantly shorter than the overall migration.

To determine how many databases will be migrated:

1. Set the INSTANCE\_URL environment variable to the URL of IBM Software Hub:

```
export INSTANCE_URL=<URL>
```

Tip: To get the URL of the web client, run the following command:

```
cpd-cli manage get-cpd-instance-details \
--cpd_instance_ns=${PROJECT_CPD_INST_OPERANDS}
```

2. Get the credentials for the wdp-service:

```
TOKEN=$(oc get -n ${PROJECT_CPD_INST_OPERANDS} secrets wdp-service-id -o yaml |
grep service-id-credentials | cut -d':' -f2- | sed -e 's/ //g' | base64 -d)
```

3. Get the number of catalogs in the instance:

```
curl -sk -X GET "https://${INSTANCE_URL}/v2/catalogs?
limit=10001&skip=0&include=catalogs&bss_account_id=999" -H 'accept:
application/json' -H "Authorization: Basic ${TOKEN}" | jq -r '.catalogs | length'
```

4. Get the number of projects in the instance:

```
curl -sk -X GET "https://${INSTANCE_URL}/v2/catalogs?
limit=10001&skip=0&include=projects&bss_account_id=999" -H 'accept:
application/json' -H "Authorization: Basic ${TOKEN}" | jq -r '.catalogs | length'
```

5. Get the number of spaces in the instance:

```
curl -sk -X GET "https://${INSTANCE_URL}/v2/catalogs?
limit=10001&skip=0&include=spaces&bss_account_id=999" -H 'accept:
application/json' -H "Authorization: Basic ${TOKEN}" | jq -r '.catalogs | length'
```

f. Add up the number of catalogs, projects, and spaces returned by the previous commands. Then, use the following table to determine approximately how long the service will be offline during the migration:

| Databases                 | Downtime for migration (approximate) |
|---------------------------|--------------------------------------|
| Up to 1,000 databases     | 6 minutes                            |
| 1,001 - 10,000 databases  | 20 minutes                           |
| 10,001 - 70,000 databases | 60 minutes                           |

g. Save the following script on the client workstation as a file named `precheck_migration.sh`:

```
#!/bin/bash

# Default ranges for couchdb size
SMALL=50
MEDIUM=100
LARGE=200

echo "Performing pre-migration checks"

patch_for_small()
{
    echo -e "Run the following command to increase the CPU and memory:\n"
    cat << EOF
oc patch ccs ccs-cr -n ${PROJECT_CPD_INST_OPERANDS} --type merge --patch '{"spec":
{
    "catalog_api_postgres_migration_threads": 4,
    "catalog_api_migration_job_resources": {
        "requests": {"cpu": "2", "ephemeral-storage": "10Mi", "memory": "2Gi"},
        "limits": {"cpu": "6", "ephemeral-storage": "1Gi", "memory": "6Gi"}}
    }
}'
EOF
    echo
    echo "The system is ready for migration. Upgrade your cluster as usual."
}

patch_for_medium()
{
    echo -e "Run the following command to increase the CPU and memory:\n"
    cat << EOF
oc patch ccs ccs-cr -n ${PROJECT_CPD_INST_OPERANDS} --type merge --patch '{"spec":
{
    "catalog_api_postgres_migration_threads": 6,
    "catalog_api_migration_job_resources": {
        "requests": {"cpu": "3", "ephemeral-storage": "10Mi", "memory": "4Gi"},
        "limits": {"cpu": "8", "ephemeral-storage": "4Gi", "memory": "8Gi"}}
    }
}'
EOF
    echo
    echo "The system is ready for migration. Upgrade your cluster as usual."
}

patch_for_large()
{
    echo -e "Run the following command to increase the CPU and memory:\n"
    cat << EOF
oc patch ccs ccs-cr -n ${PROJECT_CPD_INST_OPERANDS} --type merge --patch '{"spec":
{
    "catalog_api_postgres_migration_threads": 8,
    "catalog_api_migration_job_resources": {
        "requests": {"cpu": "6", "ephemeral-storage": "10Mi", "memory": "6Gi"},
        "limits": {"cpu": "10", "ephemeral-storage": "6Gi", "memory": "10Gi"}}
    }
}'
EOF
    echo
    echo "The system is ready for migration. Upgrade your cluster as usual."
}
```

```

}}'
EOF
    echo
    echo "Before you can start the upgrade, you must prepare the system for
migration."
}

check_resources(){
    scale_config=$1
    pvc_size=$(oc get pvc -n ${PROJECT_CPD_INST_OPERANDS} database-storage-
wdp-couchdb-0 --no-headers | awk '{print $4}')
    size=$(awk '{print substr($0, 1, length($0)-2)}' <<< "$pvc_size")

    if [[ $scale_config == "small" ]];then
        if [[ "$size" -le "$SMALL" ]];then
            echo "The system is ready for migration. Upgrade your cluster as
usual."
        elif [ "$size" -ge "$SMALL" ] && [ "$size" -le "$MEDIUM" ];then
            patch_for_medium
        elif [ "$size" -ge "$MEDIUM" ] && [ "$size" -le "$LARGE" ];then
            patch_for_large
        else
            patch_for_large
        fi
    elif [[ $scale_config == "medium" ]];then
        if [[ "$size" -le "$SMALL" ]];then
            patch_for_small
        elif [ "$size" -ge "$SMALL" ] && [ "$size" -le "$MEDIUM" ];then
            echo "The system is ready for migration. Upgrade your cluster as
usual."
        elif [ "$size" -ge "$MEDIUM" ] && [ "$size" -le "$LARGE" ];then
            patch_for_large
        else
            patch_for_large
        fi
    elif [[ $scale_config == "large" ]];then
        if [[ "$size" -le "$SMALL" ]];then
            patch_for_small
        elif [ "$size" -ge "$SMALL" ] && [ "$size" -le "$MEDIUM" ];then
            patch_for_medium
        elif [ "$size" -ge "$MEDIUM" ] && [ "$size" -le "$LARGE" ];then
            echo "The system is ready for migration. Upgrade your cluster as
usual."
        else
            patch_for_large
        fi
    fi
}

check_upgrade_case(){
    echo -e "Checking if automatic upgrade or semi-automatic upgrade is
needed"
    scale_config=$(oc get ccs -n ${PROJECT_CPD_INST_OPERANDS} ccs-cr -o json |
jq -r '.spec.scaleConfig')

```

```
# Default case, scale config is set to small
if [[ -z "${scale_config}" ]];then
    scale_config=small
fi

check_resources $scale_config
}

check_upgrade_case
```

h. Run the `precheck_migration.sh` to determine whether you can run an automatic migration of the common core services or whether you need to configure common core services to run a semi-automatic migration:

```
./precheck_migration.sh
```

Take the appropriate action based on the message returned by the script:

| Messages returned by the script                                                                                                                                                                 | Migration type | What to do next                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The system is ready for migration.<br>Upgrade your cluster as usual.                                                                                                                            | Automatic      | You are ready to upgrade IBM Software Hub.<br>Important: After you upgrade the services in your environment, ensure that you complete Completing the catalog-api service migration                                                                                                          |
| Run the following command to increase the CPU and memory.                                                                                                                                       | Automatic      | 1. Run the patch command returned by the script. 2. Upgrade IBM Software Hub. Important: After you upgrade the services in your environment, ensure that you complete Completing the catalog-api service migration                                                                          |
| The script returns both of the following messages: "Run the following command to increase the CPU and memory." + "Before you can start the upgrade, you must prepare the system for migration." | Semi-automatic | 1. Run the patch command returned by the script. 2. Run the following command to enable semi-automatic migration (아래 명령). 3. Upgrade IBM Software Hub. Important: After you upgrade the services in your environment, ensure that you complete Completing the catalog-api service migration |

반자동 활성화 명령 (판정표 3행의 것):

```
oc patch ccs ccs-cr \
-n ${PROJECT_CPD_INST_OPERANDS} \
--type merge \
--patch '{"spec": {"use_semi_auto_catalog_api_migration": true}}'
```

## About this task

Use the `cpd-cli manage install-components` command to upgrade the required operators and custom resources for an instance of IBM Software Hub.

Note: The `install-components` command in this topic includes the `--run_storage_tests` option. It is strongly recommended that you run the command with the `--run_storage_tests` option to ensure that the storage in your environment meets the minimum requirements for performance.

If your storage does not meet the minimum requirements, you can remove the `--run_storage_tests` option to continue the upgrade. However, your environment is likely to encounter problems because of issues with your storage.

## Procedure

1. Log the `cpd-cli` in to the Red Hat OpenShift Container Platform cluster:

```
${CPDM_OC_LOGIN}
```

Remember: `CPDM_OC_LOGIN` is an alias for the `cpd-cli manage login-to-ocp` command.

2. Review the license terms for the software that you plan to install. The licenses are available online. However, some licenses are not included in the `get-license` command. If you don't see the license that you purchased, you can search for the license on IBM Terms.

See all available license URLs:

```
cpd-cli manage get-license \  
--release=${VERSION}
```

See the URL for a specific license — run the appropriate commands based on the license or licenses that you purchased (Db2 만 9/3 노출, 나머지 접힘): IBM Cloud Pak for Data Enterprise Edition / IBM Cloud Pak for Data Standard Edition / IBM Data Gate for watsonx / IBM Data Product Hub Cartridge / Data Replication / Db2 / IBM Knowledge Catalog Premium / IBM Knowledge Catalog Standard / IBM watsonx.ai / IBM watsonx Code Assistant / IBM watsonx Code Assistant for Ansible / IBM watsonx.data / IBM watsonx.data Premium Edition

Db2 — run the appropriate command based on the license that you purchased: IBM Db2 Standard Edition Cartridge for IBM Cloud Pak for Data:

```
cpd-cli manage get-license \  
--release=${VERSION} \  
--license_types=DB2SE
```

IBM Db2 Advanced Edition Cartridge for IBM Cloud Pak for Data:

```
cpd-cli manage get-license \
--release=${VERSION} \
--license_types=DB2AE
```

3. Upgrade the required operators and custom resources for the instance. Tip: Before you run this command against your cluster, you can preview the oc commands that this command will issue on your behalf by running the command with the `--preview=true` option. The oc commands are saved to the `preview.sh` file in the work directory. The command that you run depends on whether the instance includes tethered projects:

Instances without tethered projects (★모비스 = 이 변형. 룰북 p.59·체크리스트 No.110 일치):

```
cpd-cli manage install-components \
--license_acceptance=true \
--components=cpd_platform \
--release=${VERSION} \
--patch_id=${PATCH_ID} \
--operator_ns=${PROJECT_CPD_INST_OPERATORS} \
--instance_ns=${PROJECT_CPD_INST_OPERANDS} \
--image_pull_prefix=${IMAGE_PULL_PREFIX} \
--image_pull_secret=${IMAGE_PULL_SECRET} \
--run_storage_tests=true \
--upgrade=true
```

Instances with tethered projects [접힘 미노출 — 모비스 해당 없음]

4. Wait for the `cpd-cli` to return the following message before preceding to the next step:

```
[SUCCESS] ... The install-components command ran successfully.
```

5. If you have any custom RSI patches that patch zen pods or IBM Cloud Pak foundational services pods, reapply the patches: a. Run the following command to get a list of the RSI patches in the operands project:

```
cpd-cli manage get-rsi-patch-info \
--cpd_instance_ns=${PROJECT_CPD_INST_OPERANDS} \
--all
```

- b. If there are patches that apply to zen or IBM Cloud Pak foundational services pods, run the following command to apply your custom patches:

```
cpd-cli manage apply-rsi-patches \
```

```
--cpd_instance_ns=${PROJECT_CPD_INST_OPERANDS}
```

6. Optional: If you want to run a batch upgrade of the services that are installed on the instance, run the `install-components` command. Tip: `--preview=true` 미리보기 가능 (`work/preview.sh`).

```
cpd-cli manage install-components \
--license_acceptance=true \
--components=${COMPONENTS} \
--release=${VERSION} \
--patch_id=${PATCH_ID} \
--operator_ns=${PROJECT_CPD_INST_OPERATORS} \
--instance_ns=${PROJECT_CPD_INST_OPERANDS} \
--image_pull_prefix=${IMAGE_PULL_PREFIX} \
--image_pull_secret=${IMAGE_PULL_SECRET} \
--upgrade=true
```

Wait for the `cpd-cli` to return the following message before preceding to the next step:

```
[SUCCESS] ... The install-components command ran successfully.
```

7. Confirm that the status of the operands is Completed:

```
cpd-cli manage get-cr-status \
--cpd_instance_ns=${PROJECT_CPD_INST_OPERANDS}
```

8. Check the health of the resources in the operators project:

```
cpd-cli health operators \
--operator_ns=${PROJECT_CPD_INST_OPERATORS} \
--control_plane_ns=${PROJECT_CPD_INST_OPERANDS}
```

Confirm that the health check report returns the expected results:

| Test                  | What the test checks                                                                                  | Expected result |
|-----------------------|-------------------------------------------------------------------------------------------------------|-----------------|
| Pod Healthcheck       | For pods in the operators project, the status of each required pod is Running.                        | [SUCCESS]       |
| Pod Usage Healthcheck | For pods in the operators project, the resource use for each pod is within the CPU and memory limits. | [SUCCESS]       |

| Test                                 | What the test checks                                                                                                                                                                                                   | Expected result |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Cluster Service Versions Healthcheck | For cluster service versions (CSVs) in the operators project, the phase of each CSV is Succeeded.                                                                                                                      | [SUCCESS]       |
| Catalog Source Healthcheck           | For catalog sources in the operators project, the last observed state of each catalog source is Ready.                                                                                                                 | [SUCCESS]       |
| Install Plan Healthcheck             | For operators in the operators project, the install plan approval for each operator is Automatic.                                                                                                                      | [SUCCESS]       |
| Subscriptions Healthcheck            | For subscriptions in the operators project, there is an installed CSV for each subscription.                                                                                                                           | [SUCCESS]       |
| Persistent Volume Claim Healthcheck  | For persistent volume claims (PVCs) in the operators project, each PVC is bound. Note: There should not be any PVC in the operators project, so the test should be skipped.                                            | [SKIP...]       |
| Deployment Healthcheck               | For deployments in the operators project, each deployment has the desired number of replicas.                                                                                                                          | [SUCCESS]       |
| Namespace Scopes Healthcheck         | For the NamespaceScope operator in the operators project, the projects that are specified in the members list exist.                                                                                                   | [SUCCESS]       |
| Stateful Set Healthcheck             | For stateful sets in the operators project, the stateful sets have the desired number of replicas. Note: There should not be any stateful sets in the operators project, so the test should be skipped.                | [SKIP...]       |
| Common Services Healthcheck          | For the common-service commonservice custom resource in the operators project, the phase of the custom resource is Succeeded.                                                                                          | [SUCCESS]       |
| Custom Resource Healthcheck          | For any other custom resources in the operators project, the phase of each custom resource is Succeeded. Note: There should not be any other custom resources in the operators project, so the test should be skipped. | [SKIP...]       |
| Operand Requests Healthcheck         | For operand requests in the operators project, the phase of each operand request is Running,                                                                                                                           | [SUCCESS]       |

9. Check the health of the resources in the operands project:

```
cpd-cli health operands \
--control_plane_ns=${PROJECT_CPD_INST_OPERANDS}
```

Confirm that the health check report returns the expected results:

| Test                                | What the test checks                                                                                                         | Expected result |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Pod Healthcheck                     | For pods in the operands project, the status of each pod is Running.                                                         | [SUCCESS]       |
| Pod Usage Healthcheck               | For pods in the operands project, the resource use for each pod is within the CPU and memory limits.                         | [SUCCESS]       |
| EDB Cluster Healthcheck             | For EDB Postgres clusters in the operands project, the status of each cluster is Cluster in healthy state.                   | [SUCCESS]       |
| Persistent Volume Claim Healthcheck | For persistent volume claims (PVCs) in the operands project, each PVC is bound.                                              | [SUCCESS]       |
| Deployment Healthcheck              | For deployments in the operands project, each deployment has the desired number of replicas.                                 | [SUCCESS]       |
| Stateful Set Healthcheck            | For stateful sets in the operands project, the stateful sets have the desired number of replicas.                            | [SUCCESS]       |
| Common Services Healthcheck         | For the common-service commonservice custom resource in the operands project, the phase of the custom resource is Succeeded. | [SUCCESS]       |
| Operand Requests Healthcheck        | For operand requests in the operands project, the phase of each operand request is Running.                                  | [SUCCESS]       |
| Monitor Events Healthcheck          | The platform monitors are not generating any Critical events.                                                                | [SUCCESS]       |
| Custom Resource Healthcheck         | For custom resources in the operands project, the phase of each custom resource is Succeeded.                                | [SUCCESS]       |
| Platform Healthcheck                | That the pods for required platform microservices are Running.                                                               | [SUCCESS]       |

## What to do next

Now that you've upgraded the IBM Software Hub control plane, you're ready to complete Updating the cpdbr service (Upgrading from Version 5.1 to Version 5.3).

If you don't use the cpdbr service, see Upgrading the IBM Software Hub configuration admission controller webhook (Upgrading from Version 5.1 to Version 5.3).

Parent topic: Upgrading an instance of IBM Software Hub Previous topic: Creating image pull secrets for an instance of IBM Software Hub Next topic: Updating the cpdbr service

Related reference: cpd-cli manage login-to-ocp / get-license / setup-instance / apply-olm / apply-cr / get-cr-status

=====

【파일】 6-인스턴스업그레이드/4-cpdbr서비스갱신.md

# 원문 — Updating the cpdbr service (5.1→5.3) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → Upgrading an instance → **Updating the cpdbr service** URL: [미확인 — 주소 받으면 기입] 문서 갱신일: 2026-08-20

★우리 판정 (런북 p.68-B10 의 원본 — 대조 결과):

- ① ★B10 뿌리 확정 – 원문은 백업 도구 4갈래(OADP/Fusion/NetApp/Portworx) 분기 구조. OADP\_OPERATOR\_NS export 단계는 ★NetApp·Portworx 갈래에만 있고 OADP 갈래엔 없음.  
런북 p.68 은 NetApp/Portworx 갈래의 export 문장 + OADP 갈래 명령을 짜깁기한 것  
– "NetApp/Portworx 쓰면 설정하라"는 조건문이 그래서 생긴 것. B10 처방(무조건 지정) 유지
- ② ★런북 p.68 과 명령 차이 발견 – 공식 OADP 갈래 플래그:  
--cpfs-image-version=latest 가 있음 (런북엔 없음) /  
런북의 --cpdbr-hooks-image-prefix·--cpfs-image-prefix (사실 주소 지정) 는 공식 5.3.x OADP 갈래에 없음 – 폐쇄망이므로 런북식 prefix 지정이 실용적일 수 있으나 [잠정] 실행판은 두 판 대조 후 결정 (5.1.x 설치구성 §3.3 은 prefix 방식)
- ③ ★스케줄러 변형 존재: --cpd-scheduler-namespace=\${PROJECT\_SCHEDULING\_SERVICE} 추가  
– 스케줄러 조건부 4번째 항목 (3종 세트 → 4종: 전역리소스·NS시크릿·apply-scheduler·cpdbr-tenant 의 스케줄러 플래그). △원문 표기 오류: 두 변형 제목이 둘 다 "Environments with the scheduling service" – 두 번째는 without 이 맞음 (명령 내용으로 판별)
- ④ Fusion/NetApp/Portworx 갈래 – 모비스 OADP → 해당 없음 (Fusion 갈래의 DPA 플러그인  
스크립트는 참고 가치: cpfs-oadp-plugin·db2u·swhub 3종 add\_plugin\_if\_missing 패턴)
- ⑤ 시점 확정: "SWH 업그레이드 후" – 런북 p.68 위치(Post Upgrade) 정당성 확인

## 본문 요지 + 명령 (OADP 갈래 — 모비스 해당)

시점: SWH(5.3.1) 업그레이드 후. 클러스터 관리자.

스케줄러 있는 환경:

```
cpd-cli oadp install \
--component=cpdbr-tenant \
--namespace=${OADP_OPERATOR_NS} \
--tenant-operator-namespace=${PROJECT_CPD_INST_OPERATORS} \
--cpd-scheduler-namespace=${PROJECT_SCHEDULING_SERVICE} \
--cpfs-image-version=latest \
--skip-recipes=true \
```

```
--upgrade=true \
--log-level=debug \
--verbose
```

스케줄러 없는 환경 (△원문 제목 오타 — 내용상 without):

```
cpd-cli oadp install \
--component=cpdbr-tenant \
--namespace=${OADP_OPERATOR_NS} \
--tenant-operator-namespace=${PROJECT_CPD_INST_OPERATORS} \
--cpfs-image-version=latest \
--skip-recipes=true \
--upgrade=true \
--log-level=debug \
--verbose
```

※ OADP\_OPERATOR\_NS: 원문 OADP 갈래엔 export 단계가 없으나 명령이 참조 — B10 처방대로 무조건 export OADP\_OPERATOR\_NS=oadp-operator 실행 (env.sh 반영됨) ※ 런북 p.68 은 --cpdbr-hooks-image-prefix=<사설>/cpopen/cpd --cpfs-image-prefix=<사설>/cpopen/cpfs 방식 — 폐쇄망 이미지 주소 지정. 실행판 결정 [잠정]: 사설 레지스트리이므로 prefix 방식(런북·5.1.x §3.3 계열) 우선, --cpfs-image-version=latest 는 :latest 폐쇄망 위험(C2 유사) 검토

(참고 — Fusion 갈래에만 있는 것: velero 파드 ephemeral-storage 2Gi/메모리 4Gi 패치 + add\_plugin\_if\_missing 스크립트로 cpfs-oadp/db2u/swhub 플러그인 정합화)

=====

## 【파일】 6-인스턴스업그레이드/5-catalog-api-완료처리.md

=====

# 원문 — Completing the catalog-api service migration (5.3.x) [2026-09-03 수집·전문]

경로: 5.3.x → Administering → Post-installation setup (Day 1) → Setting up services → **Completing the catalog-api migration** URL: <https://www.ibm.com/docs/en/software-hub/5.3.x?topic=services-completing-catalog-api-migration> 문서 갱신일: 2026-08-26 연결: 계획서 STEP 12 (No.115) — ★"생략 불가" 단계. SWH업그레이드(STEP 10) 에서 catalog-api 이사 시작 → 서비스 전부 올린 뒤 이 절차로 마무리

### ★우리 판정:

- ① ★수집 완결 – 체크리스트 No.115 "Completing the catalog-api migration" 의 실체. 이제 STEP 12 절차 전문 확보 (그동안 링크만)
- ② 자동/반자동 분기: `oc describe ccs ccs-cr | grep use_semi_auto...` → 빈 응답=자동(4번 통계로 바로) / `true`=반자동(2~3번 잡 확인·완료 후 4번). 모비스는 precheck

- 상 자동 예상(PVC 30Gi) → 대개 1.4.5.6 만
- ③ ★6단계 구조: 1방법확인 2잡상태(반자동만) 3완료patch(반자동만)  
4통계 5★PostgreSQL 백업 6★통합(consolidation)
  - ④ ★주의 – 5.6 은 "업그레이드 끝나고도 남는 후속작업":  
5 PostgreSQL 백업(pg\_dump, DB 크면 수시간) /  
6 consolidation(거버넌스 카탈로그 자산 정합화, TIMEOUT 6000s)  
→ 24시간 창 밖으로 넘어갈 수 있음 – 일정 반영 필요 [잠정]
  - ⑤ ★CouchDB 삭제(맨끝)는 "수 주 검증 후" – 당일 금지.  
되돌릴 수 없음. Python3+requests 필요 (배스천 실측 항목 추가)
  - ⑥ 스크립트 3종(migration\_status.sh·backup\_postgres.sh·  
postgres-consolidation.sh) + 청소 스크립트 2종(py.sh) –  
폐쇄망이라 미리 반출 후보 (precheck\_migration.sh 처럼)
  - ⑦ jq·python3·requests 전부 배스천에 있어야 – 실측 목록 확장

## 원문 전문

Completing the catalog-api service migration Last Updated: 2026-08-26

After you upgrade the common core services from Version 5.1 to IBM Software Hub Version 5.3, the back-end database for the catalog-api service is migrated from CouchDB to PostgreSQL.

Who needs to complete this task?

- Instance administrator

When do you need to complete this task?

- Complete this task only if: ① upgraded from Version 5.1 ② the instance includes the common core services.
- Repeat as needed (인스턴스마다)

### 1. Checking the migration method used

- 자동(automatic): CCS 가 마이그레이션 잡 완료를 기다린 뒤 관련 컴포넌트를 업그레이드
- 반자동(semi-automatic): CCS 가 컴포넌트 업그레이드와 동시에 마이그레이션 잡 실행

방법 판별:

```
oc describe ccs ccs-cr \
--namespace ${PROJECT_CPD_INST_OPERANDS} \
| grep use_semi_auto_catalog_api_migration
```

| 응답   | 유형  | 다음           |
|------|-----|--------------|
| 빈 응답 | 자동  | 4. 통계 수집으로   |
| true | 반자동 | 2. 잡 상태 확인으로 |

### 2. Checking the status of the migration jobs

(반자동만 — 자동이면 4번으로)

잡: cams-postgres-migration-job / jobs-postgres-upgrade-migration

```
oc get job cams-postgres-migration-job jobs-postgres-upgrade-migration \
--namespace ${PROJECT_CPD_INST_OPERANDS} \
-o custom-
columns=NAME:.metadata.name,STATUS:.status.conditions[0].type,COMPLETIONS:.status.
succeeded
```

- Failed → IBM Support
- InProgress → 몇 분 후 재확인
- 둘 다 Complete → 3. 완료로

### 3. Completing the migration

(반자동만)

두 잡 완료 후 PostgreSQL 마이그레이션 완료: Important: 이 단계 전 DB 업데이트 최소화 (쓰기 많으면 마이그레이션 시간 증가)

```
oc patch ccs ccs-cr \
--namespace ${PROJECT_CPD_INST_OPERANDS} \
--type merge \
--patch '{"spec": {"continue_semi_auto_catalog_api_migration": true}}'
```

CCS Completed 대기 (최소 10분, 변경 많으면 더). 확인:

```
oc get ccs ccs-cr --namespace ${PROJECT_CPD_INST_OPERANDS}
# NAME VERSION RECONCILED STATUS PERCENT AGE
# ccs-cr 11.0.0 11.0.0 Completed 100% 1d
```

- Failed → IBM Support / InProgress → 재확인 / Complete → 4번

### 4. Collecting statistics about the migration

migration\_status.sh 로 마이그레이션된 DB-자산 수 확인 (디버깅용). 클라이언트 워크스테이션에 저장:

```
#!/bin/bash

# Set postgres connection parameters
postgres_password=$(oc get secret -n ${PROJECT_CPD_INST_OPERANDS} ccs-cams-
postgres-app -o json 2>/dev/null | jq -r '.data."password"' | base64 -d)
postgres_username=cams_user
postgres_db=camsdb
postgres_migrationdb=camsdb_migration
```

```

echo -e "=====MIGRATION STATUS===== "

# Total migrated database(s)
databases=$(oc -n ${PROJECT_CPD_INST_OPERANDS} -c postgres exec ccs-cams-postgres-1 -- psql -t postgresql://$postgres_username:$postgres_password@localhost:5432/$postgres_migrationdb -c "select count(*) from migration.status where state='complete'" 2>/dev/null)
if [ -n "$databases" ];then
    databases_no_space=$(echo "$databases" | tr -d ' ')
    echo "Total catalog-api databases migrated: $databases_no_space"
else
    echo "Unable to fetch migration information for databases"
fi

# Total migrated assets
assets=$(oc -n ${PROJECT_CPD_INST_OPERANDS} -c postgres exec ccs-cams-postgres-1 -- psql -t postgresql://$postgres_username:$postgres_password@localhost:5432/$postgres_db -c "select count(*) from cams.asset" 2>/dev/null)
if [ -n "$assets" ];then
    assets_no_space=$(echo "$assets" | tr -d ' ')
    echo -e "Total catalog-api assets migrated: $assets_no_space\n"
else
    echo "Unable to fetch migration information for assets"
fi

```

```
./migration_status.sh
```

→ 5. PostgreSQL 백업으로

## 5. Backing up the PostgreSQL database

새 PostgreSQL DB 백업. backup\_postgres.sh 저장:

```

#!/bin/bash

# Make sure PROJECT_CPD_INST_OPERANDS is set
if [ -z "$PROJECT_CPD_INST_OPERANDS" ]; then
    echo "Environment variable PROJECT_CPD_INST_OPERANDS is not defined. This environment variable must be set to the project where IBM Software Hub is running."
    exit 1
fi

echo "PROJECT_CPD_INST_OPERANDS namespace is: $PROJECT_CPD_INST_OPERANDS"

# Step 1: Find the replica pod

```

```

REPLICA_POD=$(oc get pods -n $PROJECT_CPD_INST_OPERANDS -l app=ccs-cams-postgres -
o jsonpath='{range .items[?(@.metadata.labels.role=="replica")]}{.metadata.name}
{"\n"}{end}')
```

```

if [ -z "$REPLICA_POD" ]; then
    echo "No replica pod found."
    exit 1
fi

echo "Replica pod: $REPLICA_POD"

# Step 2: Extract JDBC URI from a secret
JDBC_URI=$(oc get secret ccs-cams-postgres-app -n $PROJECT_CPD_INST_OPERANDS -o
jsonpath="{.data.uri}" | base64 -d)

if [ -z "$JDBC_URI" ]; then
    echo "JDBC URI not found in secret."
    exit 1
fi

# Set path on the pod to save the dump file
TARGET_PATH="/var/lib/postgresql/data/forpgdump"

# Step 3: Run pg_dump with nohup inside the pod
oc exec "$REPLICA_POD" -n $PROJECT_CPD_INST_OPERANDS -- bash -c "
    TARGET_PATH=\"$TARGET_PATH\"
    JDBC_URI=\"$JDBC_URI\"
    echo \"TARGET_PATH is $TARGET_PATH\"
    mkdir -p $TARGET_PATH &&
    chmod 777 $TARGET_PATH &&
    nohup bash -c '
        pg_dump $JDBC_URI -Fc -f $TARGET_PATH/cams_backup.dump >
$TARGET_PATH/pgdump.log 2>&1 &&
        echo \"Backup succeeded. Please copy $TARGET_PATH/cams_backup.dump file from
this pod to a safe place and delete it on this pod to save space.\" >>
$TARGET_PATH/pgdump.log
    ' &
    echo \"pg_dump started in background. Logs: $TARGET_PATH/pgdump.log\"
"
```

※원문 스크립트 내부는 파드 안에서 nohup 사용 — 이걸 IBM 스크립트 원본 그대로 (우리 tmux 규칙은 배스천 셸 기준, 파드 내부 백그라운드는 별개)

이어서 (백업 모니터링):

```
./backup_postgres.sh
```

```

REPLICA_POD=$(oc get pods -n ${PROJECT_CPD_INST_OPERANDS} -l app=ccs-cams-postgres
-o jsonpath='{range .items[?(@.metadata.labels.role=="replica")]}{.metadata.name}
{"\n"}{end}')
```

```

oc rsh ${REPLICA_POD}
```

```
cd /var/lib/postgresql/data/forpgdump/
ls -lat
```

- 진행 중: pgdump.log 크기 증가
- 완료: "Backup succeeded. Please copy ... cams\_backup.dump"
- 실패: pgdump.log 에 에러 → IBM Support (pgdump.log 첨부)
- ★DB 크면 수시간

완료 후 안전한 곳으로 복사:

```
export POSTGRES_BACKUP_STORAGE_LOCATION=<directory>
oc cp ${REPLICA_POD}:/var/lib/postgresql/data/forpgdump/cams_backup.dump \
$POSTGRES_BACKUP_STORAGE_LOCATION/cams_backup.dump
oc rsh $REPLICA_POD rm -f /var/lib/postgresql/data/forpgdump/cams_backup.dump
```

→ 6. 통합으로

## 6. Consolidating the PostgreSQL database

거버넌스 카탈로그 전반의 동일 데이터 복사본을 단일 레코드로 통합 (동일 자산이 공통 속성 공유하도록). postgres-consolidation.sh 저장 (재시도 로직 MAX\_RETRIES=2, 자산 모니터링 TIMEOUT=6000s):

```
#!/bin/bash
#set -x

# Maximum number of retry attempts
MAX_RETRIES=2
RETRY_COUNT=0

# Function to run the consolidation process
run_consolidation() {
    local attempt=$1
    local skip_restart=$2
    echo "===== " >&2
    echo "Consolidation Attempt: $attempt of $MAX_RETRIES" >&2
    echo "===== " >&2

    # Step 1: Validating environment variables
    if [ -z "$PROJECT_CPD_INST_OPERANDS" ]; then
        echo "ERROR: Environment variable PROJECT_CPD_INST_OPERANDS is not defined
for namespace where CPD is running." >&2
        exit 1
    fi
    echo "✓ PROJECT_CPD_INST_OPERANDS namespace is: $PROJECT_CPD_INST_OPERANDS"
    >&2

    # Step 2: Find the replica pod
    REPLICA_POD=$(oc get pods -n $PROJECT_CPD_INST_OPERANDS -l app=ccs-cams-
postgres -o jsonpath='{range .items[?(@.metadata.labels.role=="replica")]}
```

```

{.metadata.name}{"\n"}{end}')
    if [ -z "$REPLICA_POD" ]; then
        echo "ERROR: No replica pod found." >&2
        exit 1
    fi
    echo "✓ Replica pod: $REPLICA_POD" >&2

    # Step 3: Extract JDBC URI from a secret
    JDBC_URI=$(oc get secret ccs-cams-postgres-app -n $PROJECT_CPD_INST_OPERANDS -
o jsonpath="{.data.uri}" | base64 -d)
    if [ -z "$JDBC_URI" ]; then
        echo "ERROR: JDBC URI not found in secret." >&2
        exit 1
    fi
    echo "✓ JDBC URI extracted successfully" >&2

    # Step 4: Mark catalogs as opt in
    oc exec "$REPLICA_POD" -n $PROJECT_CPD_INST_OPERANDS -- bash -c "psql -d
${JDBC_URI} -c \"UPDATE cams.catalog SET is_sharing_properties = 'true',
optimistic_lock_id = optimistic_lock_id + 1 WHERE id in (SELECT id FROM
cams.catalog where container_type = 'catalog' and (is_governed='true' or
subtype='ibm_data_product_catalog')) and catalog.bss_account = '999';COMMIT;\""
>&2

    # Step 5: Rolling restart of catalog-api (skip on retries)
    if [ "$skip_restart" != "true" ]; then
        oc rollout restart deployment/catalog-api -n $PROJECT_CPD_INST_OPERANDS
>&2
        oc rollout status deployment/catalog-api -n $PROJECT_CPD_INST_OPERANDS >&2
    fi

    # Step 6: Initialize content
    ICP4D_URL=$(oc get route cpd -o json | grep -i "host\"" | head -n 1 | awk -F
'"' '{print $4}')
    AUTH_TOKEN=$(oc get secret wdp-service-id -o yaml | grep "service-id-
credentials:" | head -n 1 | awk -F ":" '{print $2}' | base64 -d | xargs)
    curl -k -X PUT "https://$ICP4D_URL/v2/shared_assets/initialize_content?
bss_account_id=999" \
        -H "Authorization: Basic $AUTH_TOKEN" >&2

    # Step 7: Monitor asset consolidation (TIMEOUT 6000s, INTERVAL 30s)
    TIMEOUT=6000; INTERVAL=30; ASSET_NUM=1; start_time=$SECONDS
    while [ "$ASSET_NUM" -ne 0 ]; do
        if (( SECONDS - start_time >= TIMEOUT )); then
            echo "ERROR: Timeout reached after $TIMEOUT seconds. Exiting script."
>&2
            exit 1
        fi
        ASSET_NUM=$(oc exec "$REPLICA_POD" -n $PROJECT_CPD_INST_OPERANDS -- bash -
c "psql -d ${JDBC_URI} -c \"SELECT COUNT(resource_key) as asset_count FROM (
SELECT DISTINCT ON (resource_key) resource_key, catalog_id, asset_id from
cams.asset where is_revision='false' and state = 'available' and asset_type =
'data_asset' and set_id is NULL and metadata->>'is_branched' = 'false' and
(resource_key like '%|%' or identity_key like '%|%') and (catalog_id in (SELECT id

```

```

FROM cams.catalog where is_sharing_properties = 'true' and container_type =
'catalog' and state = 'active' and bss_account = '999')) AS distinct_assets;\"" |
head -3 | tail -1 | tr -d '[:blank:]')
    echo "    → Assets remaining: $ASSET_NUM" >&2
    if [ "$ASSET_NUM" -ne 0 ]; then sleep $INTERVAL; fi
done

# Step 8: Final verification → 남은 자산 수 stdout 반환
FINAL_ASSET_NUM=$(oc exec "$REPLICA_POD" -n $PROJECT_CPD_INST_OPERANDS -- bash
-c "psql -d ${JDBC_URI} -c \"SELECT COUNT(resource_key) as asset_count FROM (
SELECT DISTINCT ON (resource_key) resource_key, catalog_id, asset_id from
cams.asset where is_revision='false' and state = 'available' and asset_type =
'data_asset' and set_id is NULL and (resource_key like '%|%' or identity_key like
'%|%' or resource_key ~ '[0-9a-f]{8}-...' or identity_key ~ '...') and (catalog_id
in (SELECT id FROM cams.catalog where is_sharing_properties = 'true' and state =
'active' and container_type = 'catalog' and bss_account = '999')) AS
distinct_assets;\"" | head -3 | tail -1 | tr -d '[:blank:]')
    echo "$FINAL_ASSET_NUM"
}

# Main retry loop
echo "=====
echo "Starting Consolidation Script with Retry Logic"
echo "Maximum retries: $MAX_RETRIES"
echo "=====

while [ $RETRY_COUNT -lt $MAX_RETRIES ]; do
    RETRY_COUNT=$((RETRY_COUNT + 1))

    # Run consolidation and capture the remaining asset count
    # Skip restart on retry attempts (attempt > 1)
    if [ $RETRY_COUNT -gt 1 ]; then
        REMAINING_ASSETS=$(run_consolidation $RETRY_COUNT "true")
    else
        REMAINING_ASSETS=$(run_consolidation $RETRY_COUNT "false")
    fi

    # Handle empty or non-numeric values
    if [ -z "$REMAINING_ASSETS" ] || ! [[ "$REMAINING_ASSETS" =~ ^[0-9]+$ ]]; then
        echo "=====
        echo "ERROR: Could not determine remaining asset count on attempt
$RETRY_COUNT"
        echo "Received value: '$REMAINING_ASSETS'"
        echo "=====
        exit 1
    fi

    if [ "$REMAINING_ASSETS" -eq 0 ]; then
        echo ""
        echo "=====
        echo "✓ SUCCESS: Initial consolidation complete on attempt $RETRY_COUNT"
        echo "=====
        exit 0
    else

```

```

echo ""
echo "=====
echo "⚠ Initial consolidation incomplete on attempt $RETRY_COUNT"
echo "Remaining assets: $REMAINING_ASSETS"

if [ $RETRY_COUNT -lt $MAX_RETRIES ]; then
    echo "Retrying... (Attempt $((RETRY_COUNT + 1)) of $MAX_RETRIES)"
    echo "Waiting 60 seconds before retry..."
    echo "=====
    sleep 60
else
    echo "=====
    echo "⚠ Maximum retry attempts ($MAX_RETRIES) reached."
    echo "Initial consolidation incomplete after $MAX_RETRIES attempts."
    echo ""
    echo "Generating remaining_assets.csv and processing via API..."
    echo "=====

    # Get required variables
    ICP4D_URL=$(oc get route cpd -o json | grep -i "host\"" | head -n 1 |
awk -F '\"' '{print $4}')
    REPLICAPOD=$(oc get pods -n $PROJECT_CPD_INST_OPERANDS -l app=ccs-
cams-postgres -o jsonpath='{range .items[?(@.metadata.labels.role=="replica")]}
{.metadata.name}{"\n"}{end}')
    JDBC_URI=$(oc get secret ccs-cams-postgres-app -n
$PROJECT_CPD_INST_OPERANDS -o jsonpath="{.data.uri}" | base64 -d)
    AUTH_TOKEN=$(oc get secret wdp-service-id -o yaml | grep "service-id-
credentials:" | head -n 1 | awk -F ":" '{print $2}' | base64 -d | xargs)

    # Generate CSV with remaining assets (INCLUDES UUID patterns, NO
is_branched check)
    echo "Querying remaining assets..."
    oc exec "$REPLICAPOD" -n $PROJECT_CPD_INST_OPERANDS -- bash -c "psql
-d ${JDBC_URI} -t -A -F',' -c \"SELECT catalog_id, asset_id FROM ( SELECT DISTINCT
ON (resource_key) resource_key, catalog_id, asset_id from cams.asset where
is_revision='false' and state = 'available' and asset_type = 'data_asset' and
set_id is NULL and (resource_key like '%|%' or identity_key like '%|%' or
resource_key ~ '[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}' or
identity_key ~ '[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}') and
(catalog_id in (SELECT id FROM cams.catalog where is_sharing_properties = 'true'
and container_type = 'catalog' and state = 'active' and bss_account = '999')) AS
distinct_assets;\"" > remaining_assets.csv

    echo "✓ Remaining assets saved to remaining_assets.csv"

    # Process each asset via API
    echo ""
    echo "Processing remaining assets via API..."
    TOTAL_ASSETS=$(wc -l < remaining_assets.csv | tr -d '[:blank:]')
    CURRENT=0
    FAILED=0

    while IFS=',' read -r catalog_id asset_id; do
        # Skip empty lines

```

```

        if [ -z "$catalog_id" ] || [ -z "$asset_id" ]; then
            continue
        fi

        CURRENT=$((CURRENT + 1))
        echo "[${CURRENT}/${TOTAL_ASSETS}] Processing asset:
catalog_id=$catalog_id, asset_id=$asset_id"

        # Call the API for each asset
        HTTP_CODE=$(curl -k -s -o /dev/null -w "%{http_code}" -X PUT \
            "https://$ICP4D_URL/v2/shared_assets/initialize_content?
bss_account_id=999&asset_id=$asset_id&catalog_id=$catalog_id" \
            -H "Authorization: Basic $AUTH_TOKEN")

        if [ "$HTTP_CODE" -ge 200 ] && [ "$HTTP_CODE" -lt 300 ]; then
            echo "    ✓ Success (HTTP $HTTP_CODE)"
        else
            echo "    ✗ Failed (HTTP $HTTP_CODE) for asset_id: $asset_id
and catalog_id : $catalog_id"
            FAILED=$((FAILED + 1))
        fi

        # Small delay to avoid overwhelming the API
        sleep 1
    done < remaining_assets.csv

    echo ""
    echo "=====
echo "API Processing Complete"
echo "Total assets processed: $TOTAL_ASSETS"
echo "Failed: $FAILED"
echo "Successful: $((TOTAL_ASSETS - FAILED))"
echo "=====

    if [ $FAILED -gt 0 ]; then
        echo "⚠ Some assets failed to process. Check the output above for
details."
        exit 1
    else
        echo "✓ All remaining assets processed successfully"
        exit 0
    fi
fi
fi
done

```

실행:

```
chmod +x postgres-consolidation.sh
${OC_LOGIN}
./postgres-consolidation.sh
```

- ★최종 asset count = 0 이어야 성공. 0 아니면 재실행, 그래도 0 아니면 IBM Support

## What to do if the consolidation completed successfully

★consolidation 성공 후 수 주 동안 프로젝트·카탈로그·스페이스 정상 동작 확인. 그 후에야 마이그레이션 리소스 청소:

```
oc delete pod      -n ${PROJECT_CPD_INST_OPERANDS} -l app=cams-postgres-migration-
app
oc delete job      -n ${PROJECT_CPD_INST_OPERANDS} -l app=cams-postgres-migration-
app
oc delete cm       -n ${PROJECT_CPD_INST_OPERANDS} -l app=cams-postgres-migration-
app
oc delete secret   -n ${PROJECT_CPD_INST_OPERANDS} -l app=cams-postgres-migration-
app
oc delete pvc      cams-postgres-migration-pvc -n ${PROJECT_CPD_INST_OPERANDS}
```

## Cleaning up CouchDB databases

△Attention: CouchDB DB 영구 삭제 — IKC 정상(프로젝트·카탈로그·자산 전부 확인) 검증 전 금지. 되돌릴 수 없음.

전제: 클라이언트에 Python 3 + requests 모듈

```
python3 --version
python3 -m pip install requests
```

절차: couchdb\_cleanup 디렉토리 생성 후 아래 3파일 저장.

**couchdb\_route.yaml** (wdp-couchdb-svc 로 Route):

```
kind: Route
apiVersion: route.openshift.io/v1
metadata:
  labels:
    app: couchdb-route
    name: couchdb-route
spec:
  to:
    kind: Service
    name: wdp-couchdb-svc
  port:
    targetPort: https
  tls:
```

```

    termination: passthrough
    insecureEdgeTerminationPolicy: None
    wildcardPolicy: None

```

**cpd\_catalog\_api\_post\_migration\_couchdb\_cleanup.py** (v2\_ prefix DB 를 스레드풀로 삭제):

```

#!/usr/bin/env python3
# coding: utf-8

import requests
import json
import argparse
from concurrent.futures import ThreadPoolExecutor
import threading

# Thread-safe counters
successful_deletes = 0
failed_deletes = 0
counter_lock = threading.Lock()

def list_dbs(db_list, db_prefix):
    """List databases matching the prefix"""
    if not db_list:
        print("\nNo databases found matching prefix '{}'.format(db_prefix))
        return

    print("\nDatabases matching prefix '{}':\n".format(db_prefix))
    print("=" * 50)
    for db in db_list:
        print(db)
    print("=" * 50)
    print("\nTotal databases found: {}".format(len(db_list)))

def delete_db(db, auth, hostname, verify_ssl):
    """Delete a single database and update counters"""
    global successful_deletes, failed_deletes

    url = hostname + db
    response = requests.delete(url, auth=auth, verify=verify_ssl)

    with counter_lock:
        if response.status_code == 200:
            print("Successfully deleted database: {}".format(db))
            successful_deletes += 1
        else:
            print("Failed to delete database: {} (Status code: {})".format(
                db, response.status_code))
            failed_deletes += 1

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("-p", "--cloudantpassword", required=True,

```

```

        help="ci cloudant password")
parser.add_argument("-t", "--threads", type=int, default=5,
                    help="number of threads to use (default: 5)")
parser.add_argument("-H", "--hostname", required=True,
                    help="hostname of the Cloudant instance.")
parser.add_argument("-u", "--username", required=True,
                    help="username for Cloudant authentication")
parser.add_argument("-r", "--remove-cams-dbs", required=False,
                    help="delete all dbs belonging to CAMS")
parser.add_argument("-l", "--list-cams-dbs", required=False,
                    help="list all dbs belonging to CAMS")
parser.add_argument("--insecure-skip-tls-verify", action="store_true",
                    help="If true, the server's certificate will not be
checked for validity. This will make your HTTPS connections insecure")
args = parser.parse_args()

password = str(args.cloudantpassword)
hostname = args.hostname
username = args.username
remove_cams_dbs = args.remove_cams_dbs
list_cams_dbs = args.list_cams_dbs
verify_ssl = not args.insecure_skip_tls_verify

# Validate that exactly one of remove-cams-dbs or list-cams-dbs is provided
options_provided = sum([bool(remove_cams_dbs), bool(list_cams_dbs)])

if options_provided == 0:
    parser.error("Error: One of --remove-cams-dbs or --list-cams-dbs must be
provided")

if options_provided > 1:
    parser.error("Error: Cannot specify both --remove-cams-dbs and --list-
cams-dbs. Please provide only one")

# Set db_prefix to 'v2_' for CAMS databases
db_prefix = 'v2_'

# Ensure hostname ends with a slash
if not hostname.endswith('/'):
    hostname += '/'

url = hostname + "_all_dbs"
print("Cloudant instance: {}".format(url))

response = requests.get(url, auth=(username, password), verify=verify_ssl)
print("Response status: {}".format(response.status_code))

all_dbs = json.loads(response.text)

# Filter databases by prefix
db_list = [db for db in all_dbs if db.startswith(db_prefix)]

total_dbs = len(db_list)
total_filtered = len(all_dbs) - total_dbs

```

```

    print("Found {} total databases".format(len(all_dbs)))
    print("Filtered {} databases that don't start with '{}'"
prefix".format(total_filtered, db_prefix))
    print("Found {} databases matching prefix\n".format(total_dbs))

# If list-cams-dbs is set, just list the databases and exit
if list_cams_dbs:
    list_dbs(db_list, db_prefix)
    return

# Check if there are any databases to delete
if not db_list:
    print("No databases found to delete with prefix '{}'"
return

# Create thread pool and execute deletions
with ThreadPoolExecutor(max_workers=args.threads) as executor:
    # Create list of auth tuples for each task
    auth_list = [(username, password)] * len(db_list)
    hostname_list = [hostname] * len(db_list)
    verify_ssl_list = [verify_ssl] * len(db_list)

    # Map the delete_db function across all databases
    executor.map(lambda x: delete_db(*x), zip(db_list, auth_list,
hostname_list, verify_ssl_list))

    print("\nDeletion Summary:")
    print("-----")
    print("Total databases processed: {}".format(total_dbs))
    print("Successfully deleted: {}".format(successful_deletes))
    print("Failed to delete: {}".format(failed_deletes))

if __name__ == "__main__":
    main()

```

**cpd\_catalog\_api\_post\_migration\_cleanup.sh** (list\_dbs/remove\_dbs 래퍼): ※원문 주의: 래퍼가 호출하는 파이썬 파일명이 원문상 ...cleanup.py 로 되어 있으나 위 py 파일명은 ...couchdb\_cleanup.py — 현장에서 파일명 일치 확인 필요 [원문 표기]

```

#!/bin/bash

set -e

SCRIPT_DIR=$(dirname $(readlink -f "$0"))

usage() {
    cat <<EOF
This is a wrapper script to list existing catalog-api databases and remove those
databases.

```

Usage: cpd\_catalog\_api\_post\_migration\_cleanup.sh <action>

#### Arguments:

<action>      Action to perform: remove\_dbs or list\_dbs

#### Examples:

cpd\_catalog\_api\_post\_migration\_cleanup.sh remove\_dbs

cpd\_catalog\_api\_post\_migration\_cleanup.sh list\_dbs

EOF

```

    exit 1;
}

function create_couchdb_route {
    oc apply -f "${SCRIPT_DIR}/couchdb_route.yaml"
}

function run_cleanup {
    route_host=$(oc get route couchdb-route -o jsonpath='{.spec.host}')
    couchdb_url="https://$route_host"
    couchdb_user=admin
    couchdb_password=$(oc get secret wdp-couchdb -o
jsonpath='{.data.adminPassword}' | base64 -d)

    curl -k -s "$couchdb_url" > /dev/null # ensure that the URL is active

    if [ "$ACTION" == "remove_dbs" ]; then
        echo "Removing CAMS databases..."
        python3 "${SCRIPT_DIR}/cpd_catalog_api_post_migration_cleanup.py" \
            -H "$couchdb_url" \
            -u "$couchdb_user" \
            -p "$couchdb_password" \
            --remove-cams-dbs true \
            --insecure-skip-tls-verify
    elif [ "$ACTION" == "list_dbs" ]; then
        echo "Listing CAMS databases..."
        python3 "${SCRIPT_DIR}/cpd_catalog_api_post_migration_cleanup.py" \
            -H "$couchdb_url" \
            -u "$couchdb_user" \
            -p "$couchdb_password" \
            --list-cams-dbs true \
            --insecure-skip-tls-verify
    else
        echo "Invalid action: $ACTION" >&2
        exit 2
    fi
}

function delete_couchdb_route {
    oc delete route couchdb-route --ignore-not-found
}

if [ -z "$1" ]; then
    usage

```

```
fi

ACTION=$1

if [ "$ACTION" != "remove_dbs" ] && [ "$ACTION" != "list_dbs" ]; then
    echo "Error: Action must be either 'remove_dbs' or 'list_dbs'" >&2
    usage
fi

create_couchdb_route
run_cleanup
delete_couchdb_route
```

실행 (chmod +x 후):

```
${OC_LOGIN}
oc project ${PROJECT_CPD_INST_OPERANDS}
./cpd_catalog_api_post_migration_cleanup.sh list_dbs      # 삭제 대상 확인
./cpd_catalog_api_post_migration_cleanup.sh remove_dbs   # ★되돌릴 수 없음
./cpd_catalog_api_post_migration_cleanup.sh list_dbs     # 0 확인 (output should
show 0 databases)
```

Parent topic: Setting up services after install or upgrade

=====

【파일】 7-마무리-웹훅-모니터/1-웹훅-모니터-원격.md

=====

## 원문 — 웹훅·모니터·원격 로케이션 (5.1→5.3, 3장 합본) [2026-09-02 수집]

경로: 5.3.x → Upgrading from Version 5.1 → 마지막 3절 URL: [미확인 — 필요 시 기입] 문서 갱신일: 웹훅 2026-08-26 / 모니터 2026-08-20 / 원격 2026-03-11

### ★우리 판정:

- ① 셋 다 조건부 — "설치돼 있으면" 만 해당. 런북도 (Optional) 처리(p.69) — 구조 일치. 모비스 실측 3건으로 판정:
  - 웹훅: `oc get mutatingwebhookconfiguration | grep -i cpd-config`
  - 특권 모니터: 별도 프로젝트 존재 여부 (`oc get ns | grep -i monitor` 등)
  - 원격 로케이션: 모비스 단일 클러스터 → 해당 없음 확정 [Context]
- ② ★웹훅 페이지 보물 — ZEN\_VERSION 짝표 확보 (사실 레지스트리 시 필수):  
 5.3.1 Patch 12 → zen 6.4.11 ★우리 patch12 짝!  
 (P11→6.4.11 / P10·P9→6.4.9 / P8·P7→6.4.7 / P6·P5→6.4.5 / P4→6.4.4 /  
 P3→6.4.3 / P2→6.4.2 / P1→6.4.1 / GA→6.4.0 / 5.3.0→6.3.0)

- ③ 웹훅 정체 설명 수확: cpd-custom-ca-certs 시크릿을 파드에 주입하는 mutating webhook – 커스텀 인증서 공유용. 설치돼 있다면 업그레이드+재활성(enable) 2단계
- ④ 모니터: apply-privileged-monitoring-service 명령 (스케줄러 포함 변형 있음)

## 1. 웹훅 (Upgrading the configuration admission controller webhook)

설치돼 있을 때만. 사실 레지스트리면 ZEN\_VERSION 지정 필요:

```
${CPDM_OC_LOGIN}
export ZEN_VERSION=6.4.11 # ★5.3.1 Patch 12 짝 (위 표)
# install-cpd-config-ac (사실 변형 – 본문 접힘 미노출) 후:
cpd-cli manage enable-cpd-config-ac \
--cpd_instance_ns=${PROJECT_CPD_INST_OPERANDS}
```

## 2. 특권 모니터 (Upgrading privileged monitors)

설치돼 있을 때만 — 별도 프로젝트에 클러스터당 1회 설치되는 구조. `apply-privileged-monitoring-service` 로 업그레이드 (오퍼레이터 프로젝트만 / +스케줄러 프로젝트 포함 두 변형 — 본문 접힘).

## 3. 원격 물리 로케이션 (Upgrading remote physical locations)

허브(주 인스턴스) 업그레이드 후, 연결된 원격 로케이션의 공유 컴포넌트→에이전트→서비스 SW 순 갱신 (하위 3장).

★모비스 = 단일 클러스터, 원격 로케이션 없음 → 통째 해당 없음 [확정]